

Documentazione Progetto per esame  
Ingegneria del Software degli studenti  
De Meo Pierluigi e Carrese Davide

# DietiEstates25

Documentazione progetto A.A.  
2024/2025

GRUPPO:  
INGSW2425\_063

PIERLUIGI  
DE MEO  
N86004900

DAVIDE  
CARRESE  
N86004813

Docenti:  
Prof. SERGIO DI MARTINO  
Prof. LUIGI LIBERO LUCIO STARACE

---

## Lista di distribuzione

**Nome**

Prof. Sergio Di Martino

Prof. L.L.L. Starace

## Documenti di riferimento

**Numero di Protocollo****Titolo**

UNI-01-2025

Progetto-INGSW-2024-2025.pdf

## Documenti di riferimento

**Nome del documento****Versione****Data**

INGSW2425\_063

1.0

10/03/2026

# Sommario

|                                            |          |
|--------------------------------------------|----------|
| <b>Sommario</b>                            | <b>2</b> |
| <b>1. INTRODUZIONE</b>                     | <b>5</b> |
| 1.1 Contesto del progetto                  | 5        |
| 1.2. Obiettivi del Documento               | 5        |
| <b>2. DOCUMENTO DEI REQUISITI SOFTWARE</b> | <b>6</b> |
| 2.1 Scopo dei Requisiti                    | 6        |
| 2.2 Glossario                              | 6        |
| 2.3 Tabella dei Casi d'uso                 | 9        |
| 2.4 Descrizione dei Casi d'uso             | 10       |
| 2.5 Requisiti Funzionali                   | 12       |
| 2.7 Requisiti non Funzionali               | 13       |
| 2.8 Modellazione dei Casi d'uso            | 13       |
| <i>Figura 1 – Use Case Diagram</i>         | 15       |
| 2.9 Tabelle di Cockburn                    | 16       |
| 2.9.1 Piazza Offerta                       | 17       |
| 2.9.2 Ricerca Avanzata                     | 19       |
| 2.9.3 Cambio Password                      | 21       |
| 2.9.4 Visualizza Statistiche               | 23       |
| 2.9.5 Mock-ups                             | 24       |
| 2.10 Utenti Target (Personas)              | 28       |
| 2.10.1 Obiettivo                           | 28       |
| 2.10.2 Personas Individuate                | 29       |
| 2.10.3 Analisi dei Tratti Caratteristici   | 33       |
| 2.11 Diagramma Uml delle Classi di Design  | 34       |
| 2.11.1 Piazza Offerta                      | 35       |
| 2.11.2 Cambio Password                     | 36       |
| 2.11.3 Ricerca Avanzata                    | 37       |
| 2.11.4 Visualizza Statistiche              | 38       |
| 2.12 Diagrammi di Sequenza di Design       | 39       |
| 2.12.1 Pattern e Convenzioni Adottate      | 39       |
| 2.12.2 Obiettivi della Modellazione        | 40       |
| 2.12.2.1 Piazzare un Offerta               | 41       |
| 2.12.2.2 Cambio Password                   | 42       |
| 2.12.2.3 Ricerca Avanzata degli Immobili   | 43       |
| 2.12.2.4 Visualizza Statistiche            | 44       |

|                                                                                       |           |
|---------------------------------------------------------------------------------------|-----------|
| <b>3 DOCUMENTO DI DESIGN DEL SISTEMA</b>                                              | <b>45</b> |
| <b>3.1 Introduzione</b>                                                               | <b>45</b> |
| <b>3.2 Design Architetturale del Sistema</b>                                          | <b>45</b> |
| 3.2.1 Panoramica dei livelli                                                          | 45        |
| 3.2.2 Deployment e Containerizzazione                                                 | 46        |
| 3.2.3 Architettura del Back-end                                                       | 46        |
| 3.2.3.1 Panoramica del Back-end Services Layer                                        | 47        |
| 3.2.2 Analisi Alternative prese in considerazione                                     | 48        |
| <b>3.3 Dettaglio Componenti</b>                                                       | <b>49</b> |
| 3.3.1 Stack Tecnologico Front-end                                                     | 49        |
| 3.3.2 Stack Tecnologico Back-end                                                      | 49        |
| <b>3.4 Design persistenza dei dati</b>                                                | <b>51</b> |
| <b>3.5 Design della UI</b>                                                            | <b>54</b> |
| 3.5.1 Paradigmi visive e architettura informativa                                     | 54        |
| 3.5.2 Euristiche di Usabilità e Affordance                                            | 55        |
| 3.5.3 Adattività e Performance dell'Interfaccia                                       | 56        |
| 3.5.4 Coerenza e Standard                                                             | 57        |
| <b>3.6 Obiettivi di Qualità</b>                                                       | <b>61</b> |
| <b>3.7 Sicurezza</b>                                                                  | <b>63</b> |
| 3.7.1 JWT Tokens                                                                      | 63        |
| 3.7.1.1 Motivazioni della scelta                                                      | 63        |
| <b>3.8 Codice Sorgente sviluppato</b>                                                 | <b>65</b> |
| 3.8.1 Configurazione Docker e Script di Deploy                                        | 65        |
| 3.8.2 Dockerfile Front-end                                                            | 67        |
| 3.8.3 Dockerfile User-Service                                                         | 67        |
| 3.8.4 Dockerfile Property-Service                                                     | 68        |
| <b>3.9 Analisi delle Scelte Configurative e di Deploy</b>                             | <b>68</b> |
| <b>4 TESTING</b>                                                                      | <b>69</b> |
| <b>4.1 Strategia Utilizzata</b>                                                       | <b>70</b> |
| 4.1.1 Test Black-Box                                                                  | 70        |
| 4.1.2 Testing White-box                                                               | 78        |
| <b>5 VERSIONING E QUALITÀ DEL CODICE</b>                                              | <b>86</b> |
| <b>5.2 Report Qualità del codice</b>                                                  | <b>91</b> |
| <b>5.3 Expert Review</b>                                                              | <b>92</b> |
| 5.3.1 Checklist di usabilità adottata                                                 | 93        |
| 5.3.2 Esito sintetico della expert review                                             | 95        |
| 5.3.3 Considerazioni finali della expert review                                       | 96        |
| <b>5.4 Progettazione e conduzione di un esperimento con utenti reali o potenziali</b> | <b>97</b> |

|                                          |     |
|------------------------------------------|-----|
| 5.4.1 Descrizione dei soggetti coinvolti | 97  |
| 5.4.2 Definizione dei task               | 98  |
| 5.4.3 Procedura sperimentale             | 98  |
| 5.4.4 Metriche raccolte                  | 99  |
| 5.4.5 Risultati dell'esperimento         | 99  |
| 5.4.6 Analisi dei risultati              | 99  |
| 5.4.7 Considerazioni finali              | 100 |

# 1. Introduzione

## 1.1 Contesto del progetto

Il presente documento costituisce la documentazione completa del progetto DietiEstates25, sviluppato nell'ambito del corso di Ingegneria del Software per l'anno accademico 2024/2025.

Commissionato da SoftEngUniNa con l'obiettivo di realizzare una piattaforma moderna e performante per la gestione di servizi immobiliari.

## 1.2. Obiettivi del Documento

DietiEstates25 ha lo scopo di descrivere in modo completo e strutturato il sistema, includendo l'analisi dei requisiti, la progettazione architeturale e le principali scelte implementative.

In particolare, il documento si propone di:

- Fornire una descrizione chiara e verificabile dei **requisiti funzionali** e non funzionali del sistema;
- Definire gli **attori** coinvolti e le principali interazioni attraverso casi d'uso e modelli UML;
- Documentare le **scelte progettuali** adottate a livello architeturale e tecnologico;
- Supportare le fasi di sviluppo, testing e manutenzione del software;
- Costituire un riferimento per stakeholder, sviluppatori e docenti ai fini della valutazione del progetto.
- **Testing e Valutazione dell'Usabilità:** presentazione delle strategie di testing implementate e dei risultati della valutazione dell'usabilità condotta attraverso review sistematiche ed esperimenti con utenti reali.

## 2. Documento dei requisiti Software

### 2.1 Scopo dei Requisiti

Questa sezione raccoglie e formalizza i requisiti attesi dal sistema, suddividendoli in funzionali e non funzionali, insieme a vincoli, assunzioni e criteri di accettazione.

In particolare, il documento include:

- la definizione dei requisiti funzionali e non funzionali del sistema;
- l'identificazione degli attori coinvolti e delle Personas di riferimento, utilizzate per rappresentare in modo realistico i profili degli utenti finali;
- la rappresentazione dei comportamenti principali del sistema attraverso un Use Case Diagram generale, che fornisce una visione d'insieme delle interazioni tra attori e funzionalità;
- la descrizione testuale dei requisiti, strutturata in modo da garantire chiarezza, verificabilità e tracciabilità.

### 2.2 Glossario

Il glossario costituisce uno strumento essenziale per superare le barriere comunicative che emergono durante la fase di elicitazione dei requisiti.

Gli stakeholder utilizzano frequentemente un linguaggio tecnico specifico del proprio settore, dando talvolta per impliciti alcuni requisiti che invece devono essere esplicitati. Ognuno di essi possiede una conoscenza approfondita del proprio ambito, ma può avere una visione limitata o parziale delle esigenze e delle attività degli altri attori coinvolti. Inoltre, in alcuni contesti, possono emergere resistenze nel discutere apertamente determinati requisiti organizzativi e vincoli, anche a causa di dinamiche di potere interne.

Per questo motivo, il glossario assume un ruolo fondamentale e si propone di:

- definire in modo chiaro e univoco i termini tecnici e di dominio presenti nel documento;
- stabilire un linguaggio comune condiviso tra stakeholder e team di sviluppo;
- ridurre ambiguità e possibili fraintendimenti terminologici;
- supportare una comunicazione efficace lungo tutto il ciclo di vita del progetto.

| <i>Termine</i>                                 | <i>Descrizione</i>                                                                                                                                                                                                                                                                          |
|------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Agente immobiliare</b>                      | Professionista abilitato che opera nel campo della compravendita, locazione e gestione di immobili.                                                                                                                                                                                         |
| <b>Amministratore Agenzia immobiliare</b>      | Figura con privilegi di gestione all'interno di un'agenzia immobiliare, responsabile della gestione di quest'ultima.                                                                                                                                                                        |
| <b>Utente</b>                                  | Identifica la classe base da cui derivano i diversi profili (Registrato, Agente, Amministratore), ognuno caratterizzato da specifici permessi e capacità operative.                                                                                                                         |
| <b>Utente non registrato</b>                   | Attore che accede al sistema in modalità anonima. Le sue facoltà sono limitate alla consultazione pubblica degli annunci e all'esecuzione di ricerche avanzate, senza possibilità di interagire con le funzioni di persistenza                                                              |
| <b>Utente registrato</b>                       | Soggetto che ha completato la procedura di onboarding tramite e-mail o servizi di terze parti. Dispone di un'identità digitale univoca che gli consente di accedere a funzionalità avanzate come la prenotazione di visite, l'invio di offerte e la gestione del proprio profilo personale. |
| <b>Api (Application Programming Interface)</b> | Insieme di definizioni e protocolli che permettono al Front-end di comunicare con il Back-end. Nel progetto, le API seguono lo standard REST                                                                                                                                                |
| <b>Back-end</b>                                | Macro-componente del sistema responsabile della logica di business, della gestione dello stato e della persistenza dei dati.                                                                                                                                                                |
| <b>Front-end</b>                               | Interfaccia utente interattiva sviluppata in Vue.js 3. Agisce come client del sistema, gestendo la presentazione dei dati e la cattura degli input dell'utente                                                                                                                              |
| <b>Hashing</b>                                 | Processo crittografico irreversibile utilizzato per trasformare le password in stringhe alfanumeriche univoche.                                                                                                                                                                             |
| <b>Caso d'uso</b>                              | Notazione UML utilizzata per descrivere una specifica interazione tra un attore e il sistema volta a raggiungere un obiettivo prefissato. Ogni caso d'uso rappresenta un'unità funzionale della piattaforma                                                                                 |
| <b>JWT (Json Web Token)</b>                    | Standard aperto utilizzato per la creazione di token di accesso sicuri. Nel sistema, i JWT permettono di trasmettere informazioni sull'identità e sul ruolo dell'utente in modo compatto e firmato digitalmente                                                                             |



|                                      |                                                                                                                                                                                                                                  |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Docker</b>                        | Piattaforma di containerizzazione che permette di impacchettare i microservizi e le loro dipendenze in unità isolate (container). Garantisce la portabilità del sistema e la coerenza tra gli ambienti di sviluppo e produzione. |
| <b>Stateless</b>                     | Paradigma architetturale in cui il server non memorizza informazioni sulla sessione dell'utente tra una richiesta e l'altra. Ogni chiamata API deve contenere tutte le informazioni necessarie.                                  |
| <b>Issue</b>                         | Fase del ciclo di vita del token JWT, a seguito di un login corretto, viene generato e firmato il token per l'utente, inserendo al suo interno i claim necessari                                                                 |
| <b>Validation</b>                    | Processo di verifica dell'integrità e della validità di un token JWT.                                                                                                                                                            |
| <b>API Gateway</b>                   | Componente architetturale che funge da punto di ingresso unico per tutte le richieste provenienti dal client, occupandosi di routing, sicurezza e validazione delle richieste verso i servizi di back-end.                       |
| <b>Servizio (Service)</b>            | Componente software indipendente che implementa una specifica logica di business (es. gestione utenti o immobili) e comunica con altri servizi tramite API REST.                                                                 |
| <b>DTO</b>                           | Oggetto utilizzato per trasferire dati tra client e server, evitando di esporre direttamente le entità interne del sistema.                                                                                                      |
| <b>CRUD</b>                          | Insieme delle operazioni base su un sistema: Create, Read, Update, Delete.                                                                                                                                                       |
| <b>SPA (Single Page Application)</b> | Applicazione web che carica una singola pagina HTML e aggiorna dinamicamente il contenuto senza ricaricare l'intera pagina.                                                                                                      |
| <b>OAuth</b>                         | Protocollo di autorizzazione che consente agli utenti di autenticarsi tramite servizi esterni (Google, Facebook, GitHub) senza condividere le proprie credenziali.                                                               |
| <b>Container</b>                     | Ambiente isolato che include applicazione e dipendenze, garantendo portabilità e coerenza tra ambienti diversi.                                                                                                                  |
| <b>Use Case Diagram</b>              | Diagramma UML che rappresenta le interazioni tra attori e sistema, mostrando le funzionalità offerte.                                                                                                                            |
| <b>Persona</b>                       | Rappresentazione fittizia ma realistica di un utente tipo, utilizzata per guidare la progettazione del sistema in base ai bisogni degli utenti.                                                                                  |

## 2.3 Tabella dei Casi d'uso

| <i><b>Attore</b></i>         | <i><b>Caso d'uso</b></i>                                                                                                                                                                                                                                                                |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Amministratore</b>        | <ul style="list-style-type: none"> <li>○ Creare account amministratore.</li> <li>○ Creare account agente immobiliare</li> </ul>                                                                                                                                                         |
| <b>Agente immobiliare</b>    | <ul style="list-style-type: none"> <li>○ Inserire nuovi annunci</li> <li>○ Gestire prenotazioni</li> <li>○ Accesso tramite Google, email, Facebook e GitHub.</li> <li>○ Accedere alla dashboard</li> <li>○ Cambio password</li> </ul>                                                   |
| <b>Utente registrato</b>     | <ul style="list-style-type: none"> <li>○ Eseguire ricerca avanzata</li> <li>○ Accesso tramite Google, email, Facebook e GitHub.</li> <li>○ Visualizza cronologia</li> <li>○ Piazza offerta</li> <li>○ Acquisto immobile</li> <li>○ Prenota visita</li> <li>○ Cambio password</li> </ul> |
| <b>Utente non registrato</b> | <ul style="list-style-type: none"> <li>○ Crea account</li> <li>○ Eseguire ricerca avanzata</li> </ul>                                                                                                                                                                                   |

## 2.4 Descrizione dei Casi d'uso

In questo capitolo descriviamo tutti i casi d'uso che analizzando il progetto commissionato abbiamo individuato:

- **Registrazione tramite e-mail:** un utente non registrato ha la possibilità di registrarsi al sistema inserendo la sua e-mail personale e una password.
- **Registrazione tramite Facebook, Google e GitHub:** un utente non registrato ha la possibilità di registrarsi al sistema semplicemente usando servizi di terze parti quali Facebook, Google ed GitHub.
- **Creare account amministratore:** Il sistema permette ad un account con permessi di amministratore di creare un altro account con gli stessi privilegi.
- **Creare account agente immobiliare:** Il sistema permette ad un account con permessi di amministratore di creare e/o abilitare account con privilegi di agente immobiliare.
- **Inserire annunci:** Il sistema permette ad un account con permessi di agente immobiliare e/o amministratore di pubblicare nuovi annunci di immobili, specificando le caratteristiche dell'immobile come tipologia, destinazione d'uso, dimensioni, stato, accessibilità, classe energetica, numero di locali, numero di bagni e una descrizione testuale.
- **Gestire prenotazioni:** Il sistema permette ad un account con permessi di agente immobiliare e/o amministratore di visualizzare una lista di prenotazioni per tutti gli annunci pubblicati dall'agenzia immobiliare. Permette ad un agente di prendere a carico una prenotazione e di visualizzare la lista delle prenotazioni a carico ordinate in ordine cronologico.
- **Visualizza la dashboard:** Il sistema permette ad un account con permessi di agente immobiliare e/o amministratore di visualizzare una dashboard con tutte le statistiche relative all'interazione degli utenti con le inserzioni caricate dall'agenzia immobiliare di appartenenza.

- **Cambio password:** Il sistema permette a tutti gli utenti di cambiare la propria password utilizzata per autenticarsi in fase di login.
- **Eseguire ricerca avanzata:** Il sistema permette agli utenti di effettuare una ricerca avanzata filtrando gli immobili in base a varie caratteristiche indicate come posizione geografica, prezzo, dimensione, numeri di locali, numero di bagni, classe energetica, vicinanza a scuole e/o parchi.
- **Visualizzare cronologia delle inserzioni visualizzate:** Il sistema tiene traccia delle inserzioni che un utente visualizza e permette di visualizzarle come una lista.
- **Piazza offerta:** Un utente può piazzare un'offerta per acquistare un immobile ad una cifra ridotta rispetto a quella iniziale.
- **Acquisto immobile:** Un utente può acquistare direttamente un immobile tramite il sistema pagando la cifra indicata nell'annuncio oppure se ha piazzato un'offerta e questa viene accettata dall'agenzia immobiliare.
- **Prenota visita:** Un utente può richiedere di visitare un immobile indicando una data e una fascia oraria.

## 2.5 Requisiti Funzionali

La presente è la lista di tutti i requisiti individuati e assegnati al nostro gruppo:

- **RF1:** Il sistema deve permettere la creazione di un account amministratore con credenziali predefinite.
- **RF2:** L'amministratore deve poter modificare la propria password e creare altri account amministrativi o agenti immobiliari.
- **RF3:** Un account con permessi "Agente" deve poter creare un account con gli stessi permessi.
- **RF4:** Un utente deve potersi registrare tramite e-mail e password o tramite terze parti (Google, Facebook, GitHub).
- **RF5:** Il sistema deve garantire il login sicuro e il salvataggio protetto delle credenziali.
- **RF6:** Gli agenti devono poter creare delle inserzioni inserendo dettagli sugli immobili (posizione, foto, prezzo, ...)
- **RF7:** Gli utenti devono poter filtrare gli immobili per parametri multipli (prezzo, dimensione, posizione, ...)
- **RF8:** Deve essere possibile visualizzare su di una mappa interattiva i risultati di una ricerca.
- **RF9:** Al momento del caricamento di una nuova inserzione, in automatico il sistema deve riconoscere la presenza di scuole e/o trasporti nelle vicinanze.
- **RF10:** Gli agenti devono poter visualizzare una dashboard informativa contenente informazioni riguardanti visualizzazioni e insights degli annunci.
- **RF11:** Gli utenti (registrati) devono poter salvare le inserzioni preferite in una sezione "Preferiti".
- **RF12:** Il sistema permette di esportare il report in formato PDF/Excel

## 2.7 Requisiti non Funzionali

La presente è la lista di tutti i requisiti NON funzionali individuati e assegnati al nostro gruppo:

- **RNF1 - Architettura Distribuita:** Il sistema deve essere diviso in un Back-end (API REST) e un Front-end completamente indipendenti
- **RNF2 - Persistenza Dati:** Lo stato del sistema deve essere gestito centralmente dal Back-end.
- **RNF3 - Tecnologia:** È obbligatorio l'uso di linguaggi di programmazione Object-Oriented
- **RNF4 - Sicurezza:** Le credenziali devono essere salvate in modo sicuro (hashing) e la comunicazione deve essere protetta.
- **RNF5 - Performance:** La ricerca degli immobili deve essere efficiente e performante, garantendo tempi di risposta rapidi anche con molti annunci.
- **RNF6 - Usabilità:** L'interfaccia deve essere intuitiva e piacevole, permettendo all'utente di fruire dei servizi in modo rapido.
- **RNF7 – Containerizzazione:** Usiamo Docker per il deployment del Back-end

## 2.8 Modellazione dei Casi d'uso

Il Diagramma dei Casi d'Uso (Use Case Diagram) è una notazione semi-formale standardizzata UML utilizzata per rappresentare le funzionalità del sistema dal punto di vista degli utenti e delle entità esterne che interagiscono con esso.

Tale diagramma svolge un ruolo centrale nel processo di analisi dei requisiti, in quanto costituisce un elemento di collegamento tra i requisiti informali espressi dagli stakeholder e la progettazione formale del sistema. Attraverso una rappresentazione grafica sintetica, consente di chiarire le aspettative funzionali e di allineare la comprensione del sistema tra le diverse parti coinvolte.

In particolare, i diagrammi dei casi d'uso forniscono una rappresentazione ad alto livello che:

- identifica gli attori (utenti e sistemi esterni) che interagiscono con il sistema;
- descrive le principali funzionalità offerte dal sistema sotto forma di casi d'uso;
- evidenzia le relazioni tra attori e casi d'uso, incluse associazioni e dipendenze funzionali.

Questa rappresentazione grafica permette di ottenere una visione d'insieme immediata delle funzionalità del Sistema.



**Figura 1 – Use Case Diagram**



## 2.9 Tabelle di Cockburn

Queste tabelle consentono di documentare, in modo chiaro e sintetico, gli obiettivi dell'utente, il flusso degli eventi principali e alternativi, le precondizioni, le postcondizioni e le estensioni, nonché gli attori (umani o sistemi) coinvolti nell'interazione.

Nel contesto dello sviluppo di DietiEstates, tali tabelle sono state utilizzate in fase di analisi e progettazione per:

- Definire in modo condiviso i requisiti funzionali tra stakeholder, sviluppatori e analisti.
- Stabilire lo scenario standard (main flow) e prevedere eventuali flussi alternativi, eccezioni o casi limite (extensions).
- Identificare con precisione gli attori coinvolti (utenti, agenti, amministratori) e i loro ruoli rispetto alle specifiche funzionalità del sistema.
- Formalizzare precondizioni/postcondizioni, cioè lo stato iniziale e quello atteso al termine dell'uso case.
- Fornire una base oggettiva per la successiva modellazione UML (diagrammi delle classi, di sequenza, test case).

Questo approccio consente di garantire copertura dei requisiti, tracciabilità delle funzionalità implementate e una visione d'insieme sulle responsabilità del sistema e degli attori.

### 2.9.1 Piazza Offerta

|                              |                                                                                      |                                                 |                                                                 |
|------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------|-----------------------------------------------------------------|
| <b>USE CASE #1</b>           | <b>Utente piazza offerta.</b>                                                        |                                                 |                                                                 |
| <b>Goal in Context</b>       | Utente vuole creare una nuova offerta.                                               |                                                 |                                                                 |
| <b>Preconditions</b>         |                                                                                      |                                                 |                                                                 |
| <b>Success end Condition</b> | Il Sistema mostra una notifica di avvenuta registrazione dell'offerta inserita.      |                                                 |                                                                 |
| <b>Failed End Condition</b>  | Il Sistema mostra una notifica di errore di non avvenuta registrazione dell'offerta. |                                                 |                                                                 |
| <b>Primary Actor</b>         | Utente                                                                               |                                                 |                                                                 |
| <b>Trigger</b>               | Utente preme ricerca in Mockup M0                                                    |                                                 |                                                                 |
| <b>Main Scenario</b>         | <b>Step n.</b>                                                                       | <b>Utente</b>                                   | <b>System</b>                                                   |
|                              | 1                                                                                    | Clicca sul bottone "Dettagli" di una proprietà. |                                                                 |
|                              | 2                                                                                    |                                                 | Mostra la schermata della proprietà selezionata.                |
|                              | 3                                                                                    | Clicca su "Piazza un'offerta".                  |                                                                 |
|                              | 4                                                                                    |                                                 | Mostra il dialog in cui digitare la cifra che si vuole offrire. |
|                              | 5                                                                                    | Inserisce cifra e clicca su invia.              |                                                                 |
|                              | 6                                                                                    |                                                 | Mostra mockup M0 con messaggio di avvenuta registrazione.       |

|                                    |    |                                                            |                                                 |
|------------------------------------|----|------------------------------------------------------------|-------------------------------------------------|
| Extension A:<br>Cifra non inserita | 5a | Utente preme piazza offerta senza aver inserito una cifra. |                                                 |
|                                    | 6a |                                                            | Sistema mostra mockup M0 con messaggio d'errore |
|                                    | 5a | Ritorna a step 2 del main scenario                         |                                                 |

## 2.9.2 Ricerca Avanzata

|                              |                                                                                      |                                                       |                                                           |
|------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------------------|
| <b>USE CASE #1</b>           | <b>Utente piazza offerta.</b>                                                        |                                                       |                                                           |
| <b>Goal in Context</b>       | Utente vuole creare una nuova offerta.                                               |                                                       |                                                           |
| <b>Preconditions</b>         |                                                                                      |                                                       |                                                           |
| <b>Success end Condition</b> | Il Sistema mostra una notifica di avvenuta registrazione dell'offerta inserita.      |                                                       |                                                           |
| <b>Failed End Condition</b>  | Il Sistema mostra una notifica di errore di non avvenuta registrazione dell'offerta. |                                                       |                                                           |
| <b>Primary Actor</b>         | Utente                                                                               |                                                       |                                                           |
| <b>Trigger</b>               | Utente preme ricerca in Mockup M0                                                    |                                                       |                                                           |
| <b>Main Scenario</b>         | <b>Step n.</b>                                                                       | <b>Utente</b>                                         | <b>System</b>                                             |
|                              | 1                                                                                    | Clicca sul bottone "Dettagli" di una proprietà in M1. |                                                           |
|                              | 2                                                                                    |                                                       | Mostra M2.                                                |
|                              | 3                                                                                    | Clicca su "Piazza un'offerta".                        |                                                           |
|                              | 4                                                                                    |                                                       | Mostra M2.1.                                              |
|                              | 5                                                                                    | Inserisce cifra e clicca su invia.                    |                                                           |
|                              | 6                                                                                    |                                                       | Mostra mockup M0 con messaggio di avvenuta registrazione. |

|                                    |    |                                                            |                                                 |
|------------------------------------|----|------------------------------------------------------------|-------------------------------------------------|
| Extension A:<br>Cifra non inserita | 5a | Utente preme piazza offerta senza aver inserito una cifra. |                                                 |
|                                    | 6a |                                                            | Sistema mostra mockup M0 con messaggio d'errore |
|                                    | 5a | Ritorna a step 2 del main scenario                         |                                                 |

|                                                   |    |                                     |                                  |
|---------------------------------------------------|----|-------------------------------------|----------------------------------|
| Extension B: Clicca annulla durante step iniziali | 3b | Inserisce nome di città inesistente |                                  |
|                                                   | 4b |                                     | Mostra M0 con messaggio d'errore |
|                                                   | 5b | Ritorna a step 3 del main scenario  |                                  |

### 2.9.3 Cambio Password

|                              |                                                                        |                                 |                                   |
|------------------------------|------------------------------------------------------------------------|---------------------------------|-----------------------------------|
| <b>USE CASE #1</b>           | <b>Utente cambia password</b>                                          |                                 |                                   |
| <b>Goal in Context</b>       | Un utente vuole cambiare password dell'account personale.              |                                 |                                   |
| <b>Preconditions</b>         | L'utente deve avere effettuato l'accesso.                              |                                 |                                   |
| <b>Success End Condition</b> | La password viene aggiornata con il nuovo valore inserito dall'utente. |                                 |                                   |
| <b>Failed End Condition</b>  | La password non viene aggiornata o alterata.                           |                                 |                                   |
| <b>Primary Actor</b>         | Utente                                                                 |                                 |                                   |
| <b>Trigger</b>               | Utente clicca "Settings" in M0                                         |                                 |                                   |
| <b>Main Scenario</b>         | <b>Step n.</b>                                                         | <b>Utente</b>                   | <b>System</b>                     |
|                              | 1                                                                      | Clicca "Settings" in M0         |                                   |
|                              | 2                                                                      |                                 | Mostra menù a tendina             |
|                              | 3                                                                      | Preme su "Cambia Password"      |                                   |
|                              | 4                                                                      |                                 | Mostra M1                         |
|                              | 5                                                                      | Compila form e preme "Conferma" |                                   |
|                              | 6                                                                      |                                 | Mostra M0 e messaggio di conferma |

|              |    |                                            |  |
|--------------|----|--------------------------------------------|--|
| Extension 1: | 1a | Preme bottone vicino al bottone "Settings" |  |
|--------------|----|--------------------------------------------|--|

|                                                                 |    |                                                                         |                              |
|-----------------------------------------------------------------|----|-------------------------------------------------------------------------|------------------------------|
| Preme pulsante diverso da “Settings”                            | 2a |                                                                         | Mostra mockup corrispondente |
|                                                                 | 3a | Riparte da step n°1                                                     |                              |
| Extension 2: Clicca un bottone vicino “Cambia password”         | 3b | Clicca un’opzione diverso rispetto a “Cambia Password”                  |                              |
|                                                                 | 4b |                                                                         | Mostra mockup corrispondente |
|                                                                 | 5b | Riparte da step n°1                                                     |                              |
| Extension 3: Sbaglia inserimento vecchia password               | 5c | Digita vecchia password errata e preme pulsante “Conferma”              |                              |
|                                                                 | 6c |                                                                         | Mostra messaggio d’errore    |
|                                                                 | 7c | Riparte da step n°5                                                     |                              |
| Extension 4: Nuova password e conferma nuova password diverse   | 5d | Digita nuova password e conferma della nuova password ma sono diverse   |                              |
|                                                                 | 6d |                                                                         | Mostra messaggio d’errore    |
|                                                                 | 7d | Ritorna n°5                                                             |                              |
| Extension 5: Nuova password non conforme al minimo di sicurezza | 5e | Digita nuova password ma questa non è conforme alle norme di sicurezza. |                              |
|                                                                 | 6e |                                                                         | Mostra messaggio d’errore    |
|                                                                 | 7e | Ritorna allo step n°5                                                   |                              |

## 2.9.4 Visualizza Statistiche

| USE CASE #4                                                                    | Agente Immobiliare accede alle statistiche                                                                                                           |                                            |                                |
|--------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|--------------------------------|
| Goal in Context                                                                | Un agente immobiliare vuole visionare le proprie statistiche                                                                                         |                                            |                                |
| Preconditions                                                                  | L'agente immobiliare deve avere effettuato il login all'applicativo.                                                                                 |                                            |                                |
| Success End Condition                                                          | Il Sistema tiene traccia di ogni operazione dell'agente immobiliare; L'agente riceve il report di tutte le sue operazioni effettuate.                |                                            |                                |
| Failed End Condition                                                           | Il Sistema non tiene traccia delle operazioni dell'agente immobiliare; il sistema tiene t traccia del fallimento; nessuna statistica viene mostrata. |                                            |                                |
| Primary Actor                                                                  | Agente immobiliare                                                                                                                                   |                                            |                                |
| Trigger                                                                        | Agente immobiliare clicca su "visualizza statistiche" in M0.1                                                                                        |                                            |                                |
| Main Scenario                                                                  | Step n.                                                                                                                                              | Utente                                     | System                         |
|                                                                                | 1                                                                                                                                                    | Clicca sull'immagine nel menù utente in M0 |                                |
|                                                                                | 2                                                                                                                                                    |                                            | Mostra M0.1                    |
|                                                                                | 3                                                                                                                                                    | Clicca su visualizza statistiche           |                                |
|                                                                                | 4                                                                                                                                                    |                                            | Mostra M3 (con risultati)      |
| Extension A:<br>L'agente clicca su un o pulsante al posto menù <u>utente</u> . | 1a                                                                                                                                                   | Preme un pulsante cino al menù utente.     |                                |
|                                                                                |                                                                                                                                                      |                                            | Mostra mock-up corrispondente. |
|                                                                                | 2a                                                                                                                                                   | Ritorna a step 1 del ain scenario          |                                |
| Extension B: Clicca annulla/esci durante elezione in Mo.1                      | 3b                                                                                                                                                   | Clicca "Annulla"                           |                                |
|                                                                                | 4b                                                                                                                                                   |                                            | Mostra mock-up corrispondente. |
|                                                                                | 5b                                                                                                                                                   | Ritorna a step 1 del ain scenario          |                                |

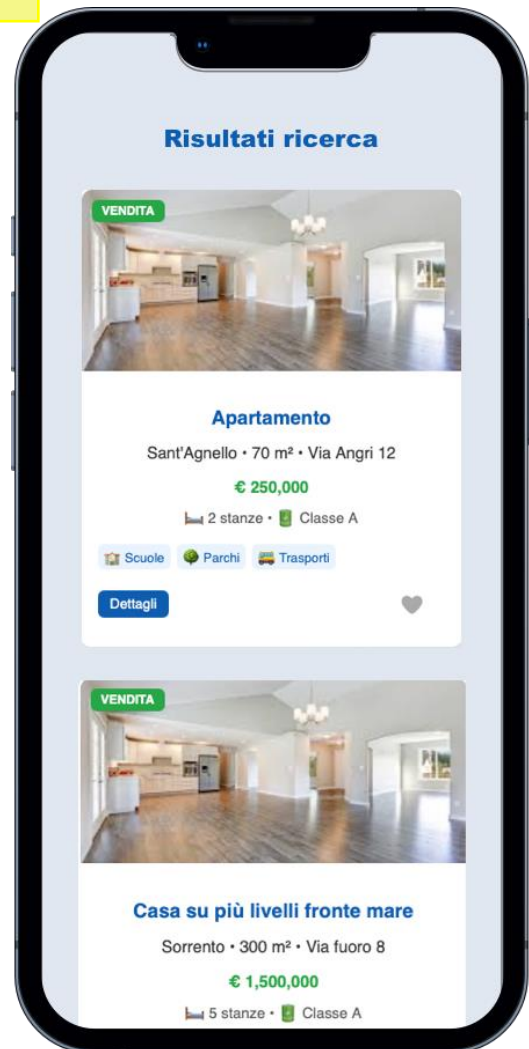


## 2.9.5 Mock-ups

Visibile solo quando l'utente ha effettuato il login, bottone per pagina login altrimenti



Mockup M0



Mockup M1



Mockup M3

Pagina visibile solo  
ad utente con  
responsabilità  
“Admin” o  
“Agente”

Profilo

Admin

Pubblica

Insights

Logout

Esplora immobili →

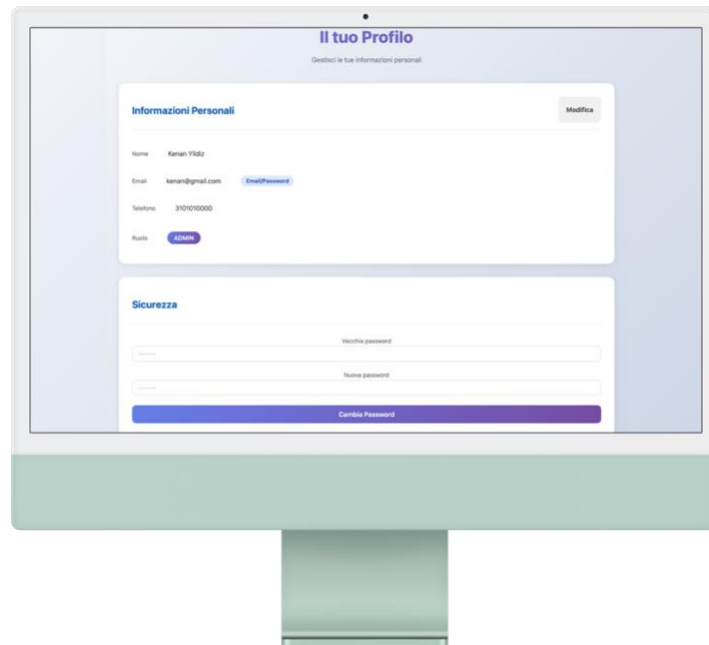
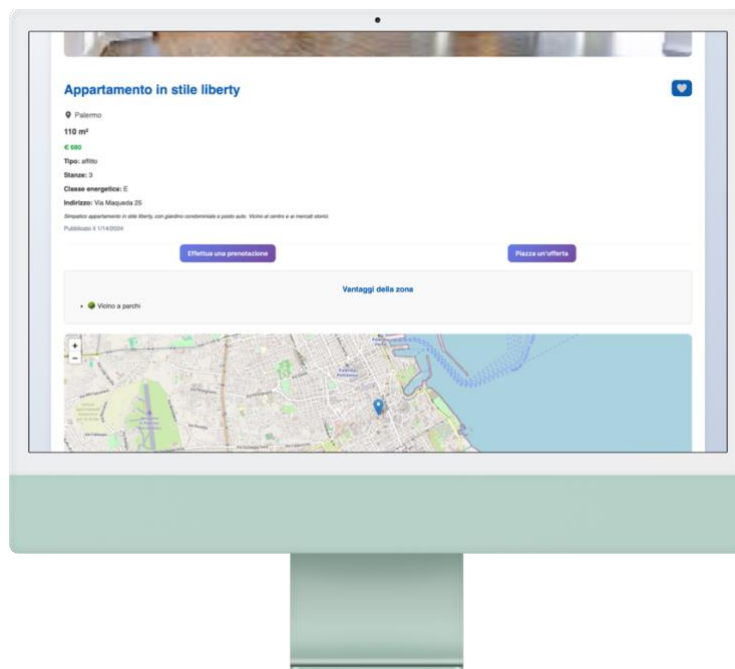
Pubblica annuncio

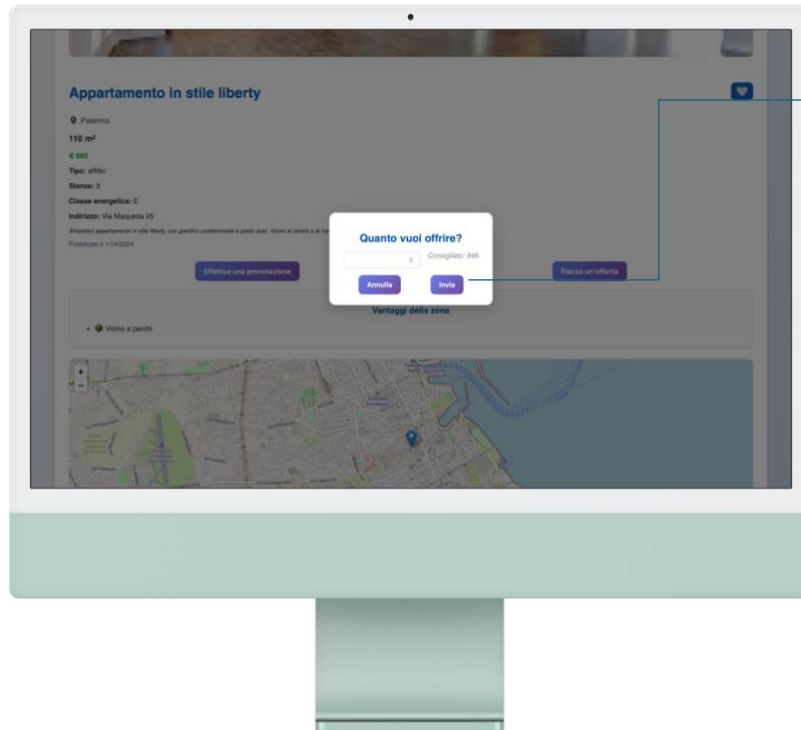
### Trova la tua casa ideale

|                      |                      |
|----------------------|----------------------|
| Città                | Prezzo               |
| <input type="text"/> | <input type="text"/> |
| Superficie (mq)      | Tipologia            |
| <input type="text"/> | <input type="text"/> |
| Stanze               | Classe energetica    |
| <input type="text"/> | <input type="text"/> |

[Cerca](#)

Mockup M0.1

*Mockup M4**Mockup M2*



Modal a comparsa dopo la pressione su "Piazza un'offerta"

Mockup M2.1

## 2.10 Utenti Target (Personas)

Nello sviluppo del software, è essenziale avere un'idea concreta dell'utente che utilizza l'applicazione. Questo aiuta il team di sviluppo a concentrarsi su determinati tipi di utenti, categorie ben specifiche che riassumono idee, obiettivi, ruoli e ambizioni personali. La sintesi di un cluster ben preciso di utenti è chiamata personas, ottenute attraverso ricerche di mercato e analisi comportamentali. Ogni persona sintetizza comportamenti, obiettivi e caratteristiche rilevanti di utenti reali che utilizzeranno il software da progettare.

### 2.10.1 Obiettivo

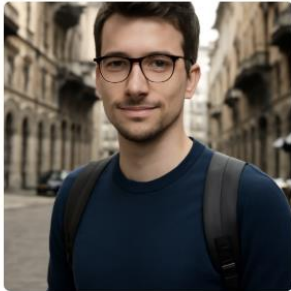
Nello specifico, le personas permettono ai progettisti di concentrarsi sulle esigenze reali dei futuri utenti del portale immobiliare, evitando generalizzazioni troppo ampie. In altre parole, permettono di focalizzare l'attenzione solamente su un gruppo ristretto di persone con determinate caratteristiche, evitando la creazione di un prodotto che cerchi di soddisfare un pubblico troppo variegato, finendo inevitabilmente all'insuccesso. Prima di passare alle personas individuate, bisogna seguire dei passi ben specifici: innanzitutto, stabilire le variabili comportamentali, ovvero tutte le caratteristiche osservabili e misurabili dell'utente come, ad esempio, la tipologia di ricerca immobiliare svolta, la frequenza di consultazione dei portali, le modalità di contatto preferite (online vs agenzia fisica), attitudini verso la tecnologia, motivazioni d'acquisto e capacità digitali. Dopo l'individuazione di tali variabili, è necessario far emergere quelle più importanti ai fini dello sviluppo software. In conclusione, una volta riassunti i dati e lo studio fatto sugli utenti, è possibile sviluppare queste personas partendo dalle informazioni raccolte.

### 2.10.2 Personas Individuate

Per la creazione delle Personas abbiamo consultato varie fonti da cui sono emersi i dati rilevanti per la costruzione di questi modelli: l'Indagine Tecnoborsa 2024 sui comportamenti di acquirenti e venditori immobiliari italiani, i dati demografici pubblicati da MutuiOnline.it (2024) e AWN.it (2024) relativi all'età media degli acquirenti, e l'Osservatorio Portali Immobiliari di CoachImmobiliare.it (2023) che analizza i canali digitali più utilizzati dagli italiani nella ricerca casa. Ulteriori insight sono stati estratti dalle analisi di Semrush (2025) sul traffico dei principali portali immobiliari italiani.

Di seguito riportiamo le Personas che sono emerse facendo inferenza sui dati raccolti dalle fonti sopra citate.

## Marco Giovane Professionista



"La mia casa ideale è un luogo dove posso rilassarmi dopo una giornata di lavoro e costruire il mio futuro."

**Age:** 29

**Gender:** Male

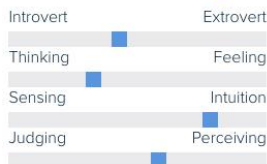
**Work:** Software Developer

**Family:** Single,

**Location:** Milano, Italia

**Character:** Ambizioso, pragmatico

### Personality



Determinato

Riflessivo

Risoluto

Pragmatico

### Goals

- Acquistare la sua prima casa, un bilocale o trilocale a Milano con budget di €200.000-250.000.
- Stabilire una base solida per la vita adulta e la carriera.
- Vivere in un ambiente moderno e ben collegato ai servizi e al lavoro.

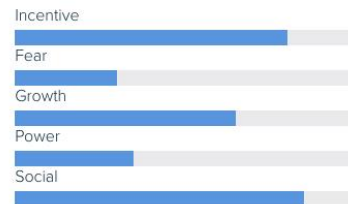
### Frustrations

- Prezzi immobiliari elevati a Milano che limitano le opzioni disponibili.
- Difficoltà nel trovare soluzioni abitative moderne con un buon rapporto qualità-prezzo.
- Processi burocratici complessi e mancanza di trasparenza nel settore immobiliare.

### Bio

Marco Rossi è un giovane sviluppatore software di Milano con un reddito annuale di circa €35.000. Dopo alcuni anni di carriera in crescita nel settore tecnologico, ha deciso di investire in una casa propria per sentirsi più stabile e indipendente. Ama la vita urbana ma cerca un equilibrio tra comfort, modernità e praticità. È attento ai dettagli, valuta attentamente le opzioni e si informa online prima di prendere decisioni importanti, rendendolo un potenziale acquirente informato e consapevole.

### Motivation



### Brands & Influencers

Marco segue brand tecnologici e lifestyle come Apple, Xiaomi e IKEA, apprezza i servizi digitali innovativi come Spotify e Revolut, e si ispira a figure del mondo tech come Elon Musk e imprenditori italiani del settore digitale. È influenzato da community online di developer e da canali YouTube che trattano di finanza personale e real estate per giovani professionisti.



### Preferred Channels



## Laura Insegnante



"Voglio una casa dove mio figlio possa crescere sereno e noi possiamo finalmente respirare."

**Age:** 38 anni

**Gender:** Femmina

**Work:** Insegnante

**Family:** Sposata, un figlio di 5 anni

**Location:** Roma, periferia

**Character:** Pragmatica, attenta e orientata alla famiglia

Responsabile

Riflessiva

Pratica

Affidabile

### Goals

- Vendere l'attuale appartamento in modo rapido e sicuro.
- Acquistare una casa più grande con giardino o terrazzo in zona familiare e tranquilla.
- Ottenere supporto professionale per gestire serenamente il doppio processo di vendita e acquisto.

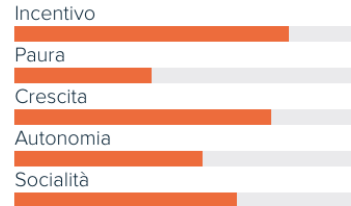
### Frustrations

- Non riesce a capire se le zone sono davvero adatte a famiglie con bambini.
- Ha paura di truffe o procedure poco trasparenti nel mercato immobiliare.
- Ha poco tempo da dedicare alle visite e trova complesso gestire contemporaneamente la vendita e l'acquisto.

### Bio

Laura ha 38 anni, insegna in una scuola primaria di Roma e vive con il marito e il figlio di cinque anni. Con un reddito familiare di circa 55.000 € l'anno, sente il bisogno di più spazio e di un ambiente più adatto alla crescita del bambino. Sta cercando una nuova casa di quattro locali con giardino o terrazzo nella periferia romana, con un budget tra 300.000 e 350.000 €. Ha competenze digitali intermedie e utilizza soprattutto lo smartphone per cercare immobili sui portali online, ma si affida anche al passaparola e alla consulenza di agenzie per le decisioni importanti. Desidera vivere in una zona sicura, con scuole e servizi, dove la famiglia possa radicarsi e crescere serenamente.

### Motivation



### Brands & Influencers

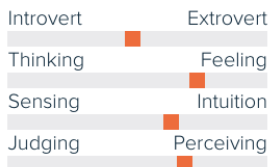
Laura si fida dei grandi portali immobiliari come Immobiliare.it e Casa.it, ma segue anche pagine Facebook e gruppi locali dove le famiglie condividono esperienze sui quartieri di Roma. Apprezza i consigli di agenti immobiliari trasparenti e di influencer che parlano di vita familiare e gestione domestica con toni autentici e concreti.

B

B



### Personality



### Preferred Channels





## Giuseppe | Imprenditore



"Investire in mattoni è come costruire sicurezza, un passo alla volta."

**Età:** 45-54 anni

**Genere:** Maschile

**Lavoro:** Imprenditore fiorentino

**Famiglia:** Sposato, figli adulti o prossimi all'indipendenza

**Località:** Firenze, Toscana

**Carattere:** Pragmatico e analitico, con approccio razionale agli investimenti

Analitico

Prudente

Orientato al rendimento

Digitale curioso

## Obiettivi

- Individuare una seconda casa in zona turistica o universitaria da mettere a reddito con ROI tra 4% e 6%.
- Creare rendite passive come supporto per la pensione e per diversificare il patrimonio.
- Investire in modo consapevole, basandosi su dati reali e analisi di mercato affidabili.

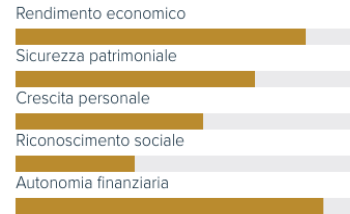
## Frustrazioni

- Difficoltà nel valutare il reale potenziale di rendita degli immobili in zone turistiche o universitarie.
- Mancanza di dati trasparenti su affitti, domanda e flussi turistici locali.
- Diffidenza verso annunci eccessivamente promozionali e poca fiducia in valutazioni non supportate da report dettagliati.

## Biografia

Giuseppe è un imprenditore di Firenze con un reddito annuo superiore a 80.000 €, che desidera ampliare il proprio portafoglio investendo in immobili. Dopo anni di lavoro nel settore privato, ora punta alla stabilità economica attraverso il mercato immobiliare, cercando una seconda casa nelle aree turistiche o universitarie. Con competenze digitali di livello base-intermedio, utilizza calcolatori di ROI online e si informa tramite riviste e agenzie di fiducia. È un investitore attento, motivato dalla volontà di costruire una rendita passiva e garantire un futuro sereno.

## Motivazioni



## Brand e influencer

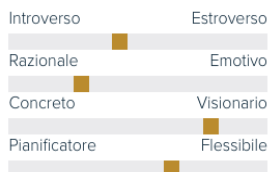
Giuseppe segue riviste come "Il Sole 24 Ore Casa24" e portali immobiliari come Immobiliare.it e Idealista. Si fida di esperti del settore immobiliare, consulenti finanziari e agenzie locali con esperienza e trasparenza nelle valutazioni.

24

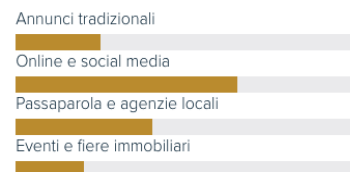
B

idealista

## Personalità



## Canali preferiti



### 2.10.3 Analisi dei Tratti Caratteristici

Nel dettagliato esame delle personas, emergono distinti tratti che intendiamo identificare in modo univoco attraverso attributi chiave:

**1 - Il Giovane Professionista - Marco Rossi:** Questa categoria è incarnata dalla persona di "Marco Rossi". L'attributo distintivo di questa tipologia di utente è la sua forte propensione digitale e il desiderio di autonomia nel processo di ricerca. Pertanto, si rivelerebbe essenziale implementare nel sistema funzionalità self-service avanzate per facilitare il suo approccio, come filtri di ricerca intelligenti, comparazione rapida tra immobili, mappe interattive con layer informativi (trasporti, servizi, zone) e sistema di notifiche personalizzate.

**2 - Famiglia in Crescita - Laura Bianchi:** Questa categoria di utenti, personificata da "Laura Bianchi," rappresenta coloro che necessitano di informazioni contestuali e supporto decisionale. Al fine di agevolare questo specifico target, si è pensato di introdurre filtri family-oriented come vicinanza scuole o parchi. Per ridurre l'ansia decisionale, si propone l'integrazione di appuntamenti online con agenti immobiliari direttamente dalla piattaforma, ottimizzando i tempi per chi ha famiglia.

**3 - L'Investitore Maturo - Giuseppe Ferrara:** Questa tipologia di utenti è rappresentata da "Giuseppe Ferrara" e comprende coloro che utilizzano la piattaforma per finalità di investimento immobiliare. Si tratta di individui interessati a dati analitici, rendimenti e proiezioni finanziarie. Noi personalmente abbiamo gestito questo tipo di personas tramite la possibilità di piazzare offerte sulle case e una dashboard affidabile, ciò non toglie in futuro di poter perfezionare l'adattamento a questa categoria tramite una dashboard ancora più sofisticata che permette: storico prezzi zona negli ultimi 5 anni, prezzo medio affitti zona, occupancy rate stimato, ROI calcolato automaticamente, domanda turistica/studentesca dell'area.

## 2.11 Diagramma Uml delle Classi di Design

I diagrammi delle classi rappresentano la struttura statica del sistema, mostrando le classi che lo compongono, i loro attributi, le operazioni (metodi) e le relazioni tra di esse. Questi diagrammi costituiscono la base di qualsiasi soluzione software orientata agli oggetti e fungono da ponte tra l'analisi dei requisiti e l'implementazione concreta.

- Nel contesto di DietiEstates, i diagrammi delle classi seguono il pattern architetturale MVC (Model-View-Controller) integrato con una three-tier architecture (Presentation, Business Logic, Data Access), articolati nei seguenti livelli:
- Boundary (Presentation Layer): Componenti Vue.js che gestiscono l'interfaccia utente e l'interazione con l'utente finale
- Control (Controller Layer): Classi Spring Boot che ricevono le richieste HTTP, orchestrano le operazioni e restituiscono le risposte
- Service (Business Logic Layer): Classi che contengono la logica di business applicativa, validazioni e trasformazioni dati
- Repository (Data Access Layer): Interfacce JPA che astraggono l'accesso al database
- Entity (Persistence Layer): Classi che mappano le tabelle del database mediante ORM (Object-Relational Mapping)
- DTO (Data Transfer Objects): Oggetti di trasferimento dati che disaccoppiano la rappresentazione interna delle entità da quella esposta alle API REST

Questa stratificazione garantisce separation of concerns, facilitando la manutenibilità, la testabilità e l'evoluzione del sistema.

## 2.11.1 Piazza Offerta

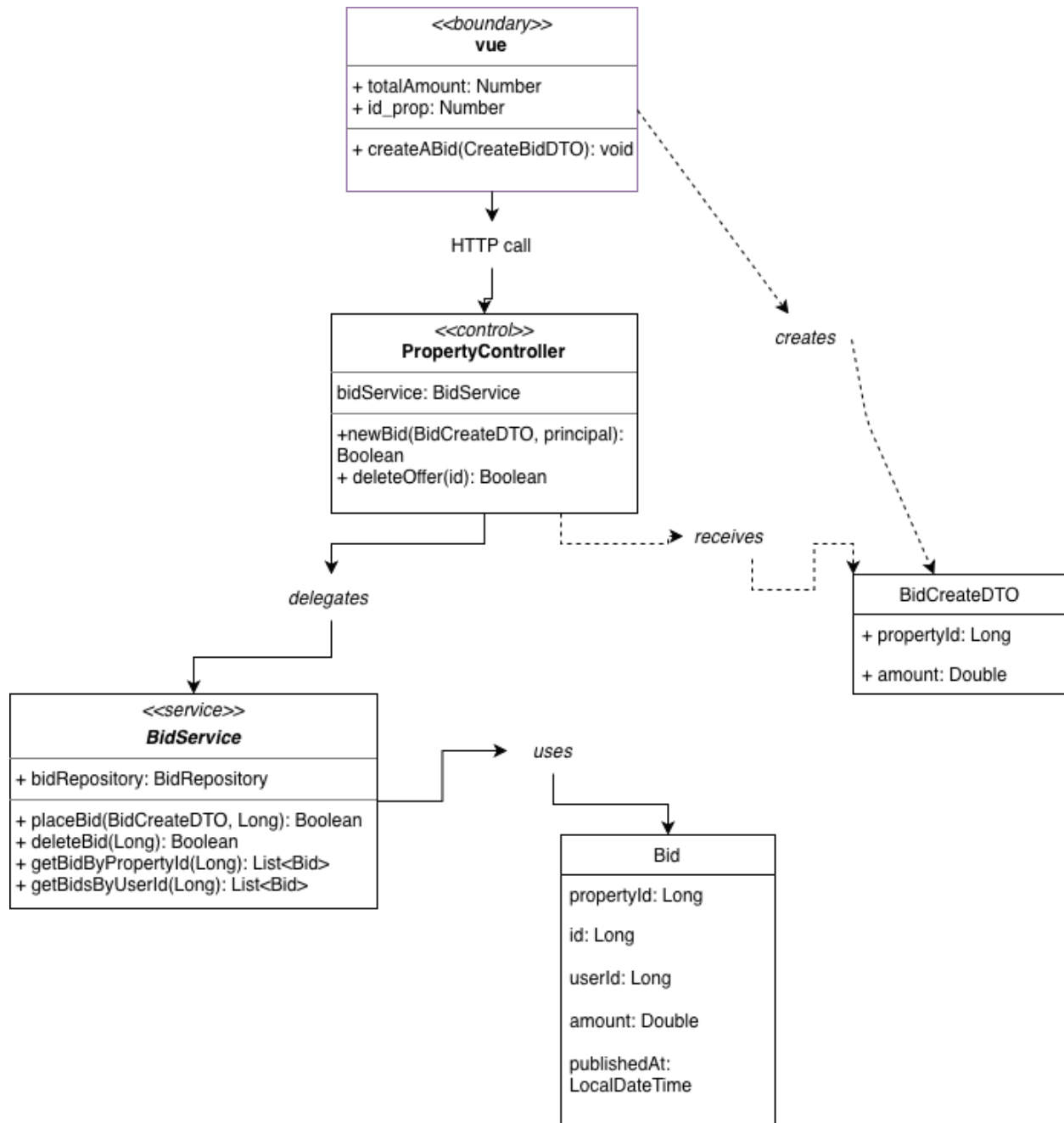


Figura 2 – Uml “Piazza Offerta”

## 2.11.2 Cambio Password

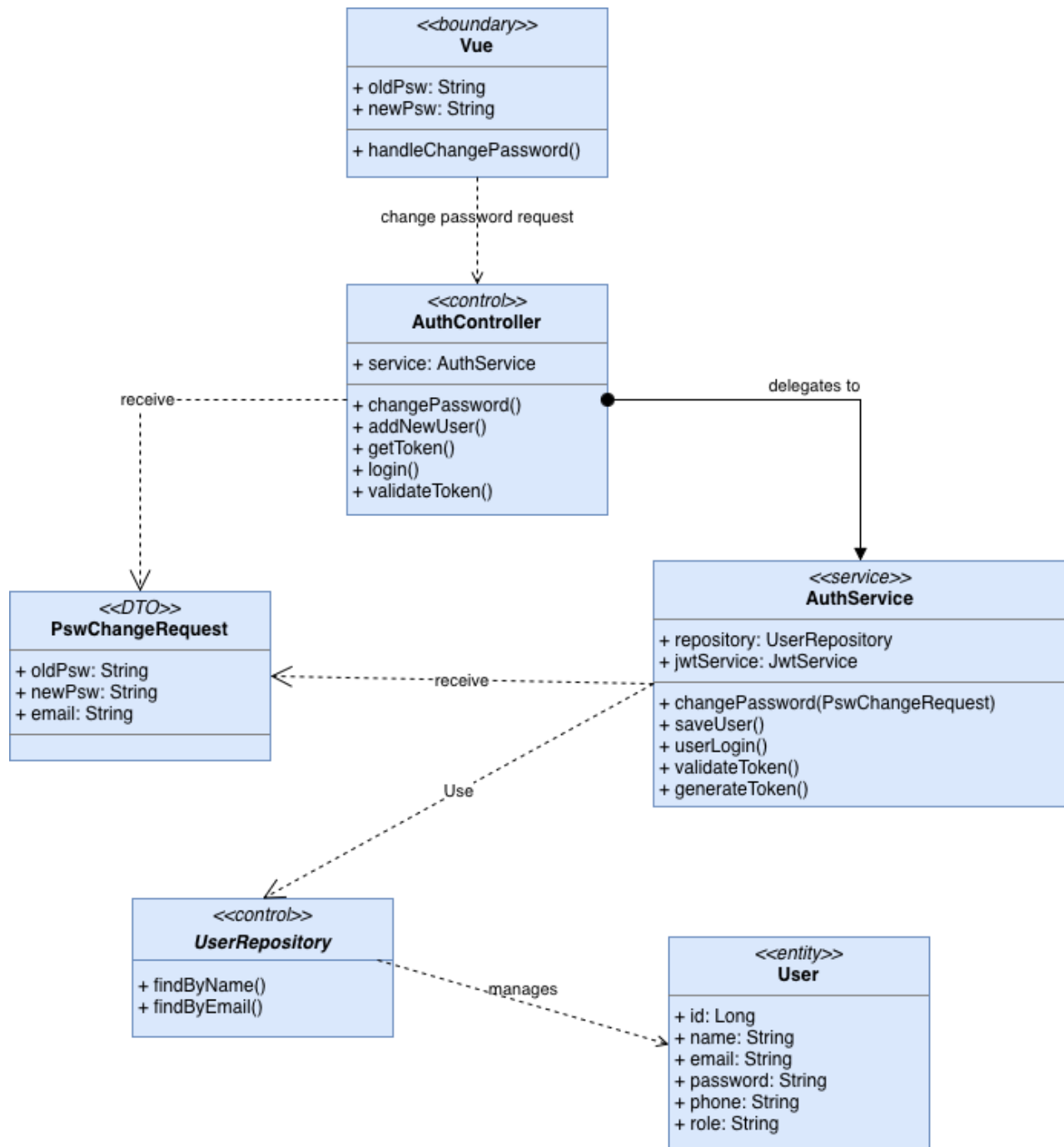


Figura 3 – Uml “changePassword”

## 2.11.3 Ricerca Avanzata

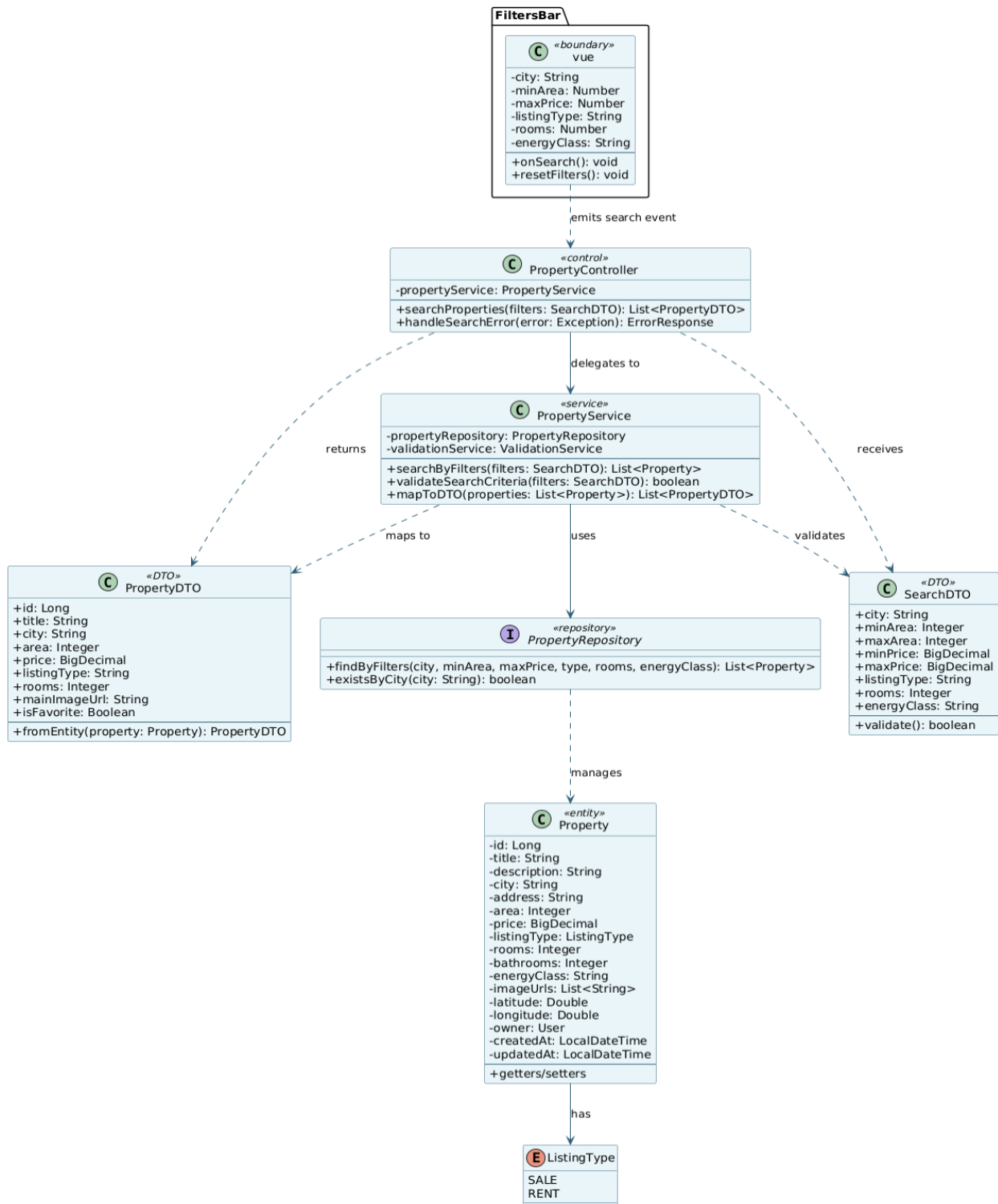


Figura 4 – Uml “Ricerca Avanzata”

## 2.11.4 Visualizza Statistiche

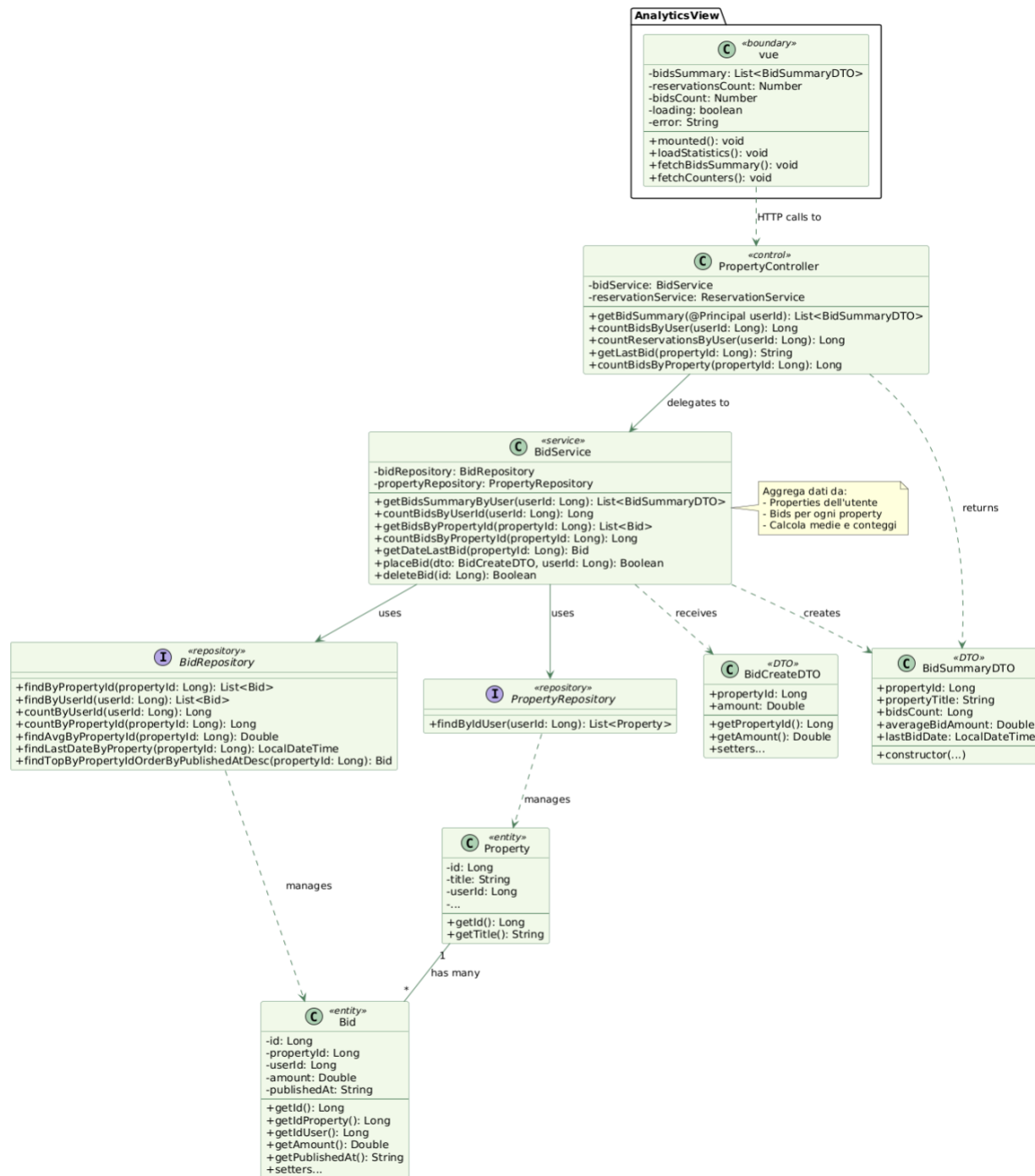


Figura 5 – Uml “Visualizza Statistiche”

## 2.12 Diagrammi di Sequenza di Design

I diagrammi di sequenza catturano gli aspetti dinamici del sistema, mostrando come gli oggetti interagiscono tra loro nel tempo attraverso lo scambio di messaggi. Ogni diagramma di sequenza illustra un particolare scenario d'uso, evidenziando l'ordine temporale delle chiamate di metodo, i dati scambiati e i flussi alternativi (estensioni).

Per DietiEstates, i diagrammi di sequenza documentano:

- Il flusso principale delle operazioni (main scenario)
- Le precondizioni necessarie per l'esecuzione dello scenario
- Le interazioni tra frontend (Vue.js) e backend (Spring Boot) tramite API REST
- Le chiamate asincrone al database PostgreSQL
- I flussi di errore e le estensioni che gestiscono casi limite

Questi diagrammi risultano particolarmente utili per comprendere il ciclo di vita completo di una richiesta utente, dalla presentazione alla persistenza dei dati, includendo validazioni, trasformazioni e gestione degli errori.

### 2.12.1 Pattern e Convenzioni Adottate

Nei diagrammi presentati sono stati applicati i seguenti pattern e convenzioni:

- Repository Pattern: Astrazione dell'accesso ai dati tramite interfacce JPA
- Service Layer Pattern: Centralizzazione della logica di business in classi dedicate
- DTO Pattern: Separazione tra modello di dominio interno e rappresentazione API esterna
- Dependency Injection: Gestione delle dipendenze tramite Spring Framework



- Gli stereotipi UML (<<boundary>>, <<control>>, <<service>>, <<repository>>,
  - <<entity>>, <<DTO>>) identificano chiaramente il ruolo di ciascuna classe nell'architettura
- Le frecce di associazione indicano la direzione delle dipendenze e delle chiamate
- Le molteplicità (1, \*, 0..1) specificano le cardinalità delle relazioni
- Le note forniscono dettagli implementativi rilevanti (query SQL, validazioni, trasformazioni)

### 2.12.2 Obiettivi della Modellazione

La documentazione tramite diagrammi UML persegue i seguenti obiettivi:

1. **Comunicazione:** Fornire un linguaggio visuale comune comprensibile a tutti i membri del team, indipendentemente dal ruolo
2. **Documentazione:** Creare una base documentale di riferimento per sviluppi futuri e manutenzione
3. **Analisi:** Identificare potenziali problemi architetturali in fase di design, prima dell'implementazione
4. **Validazione:** Verificare che l'architettura implementata rispetti i principi SOLID e le best practices
5. **Onboarding:** Facilitare l'inserimento di nuovi sviluppatori nel progetto fornendo una visione d'insieme chiara

## 2.12.2.1 Piazzare un Offerta

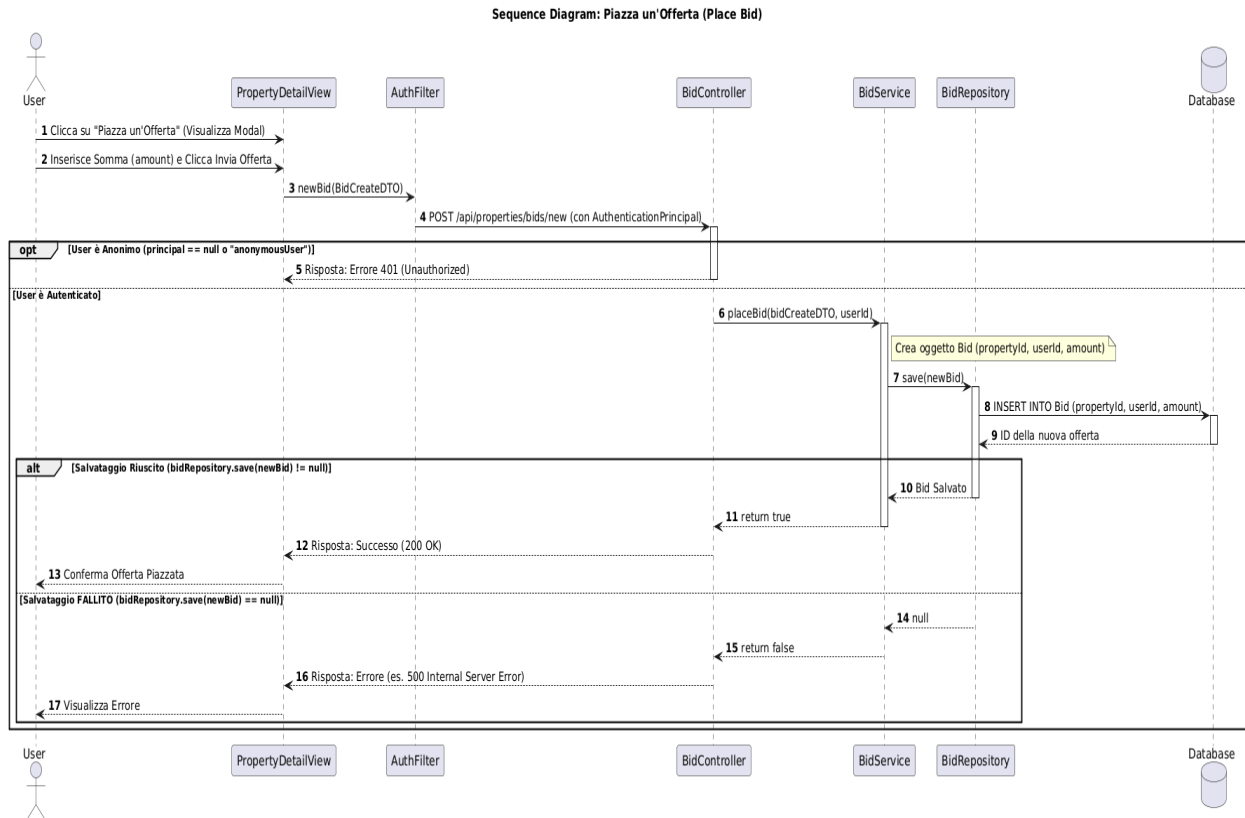


Figura 6 – Statechart “Piazzare un’offerta”

## 2.12.2.2 Cambio Password

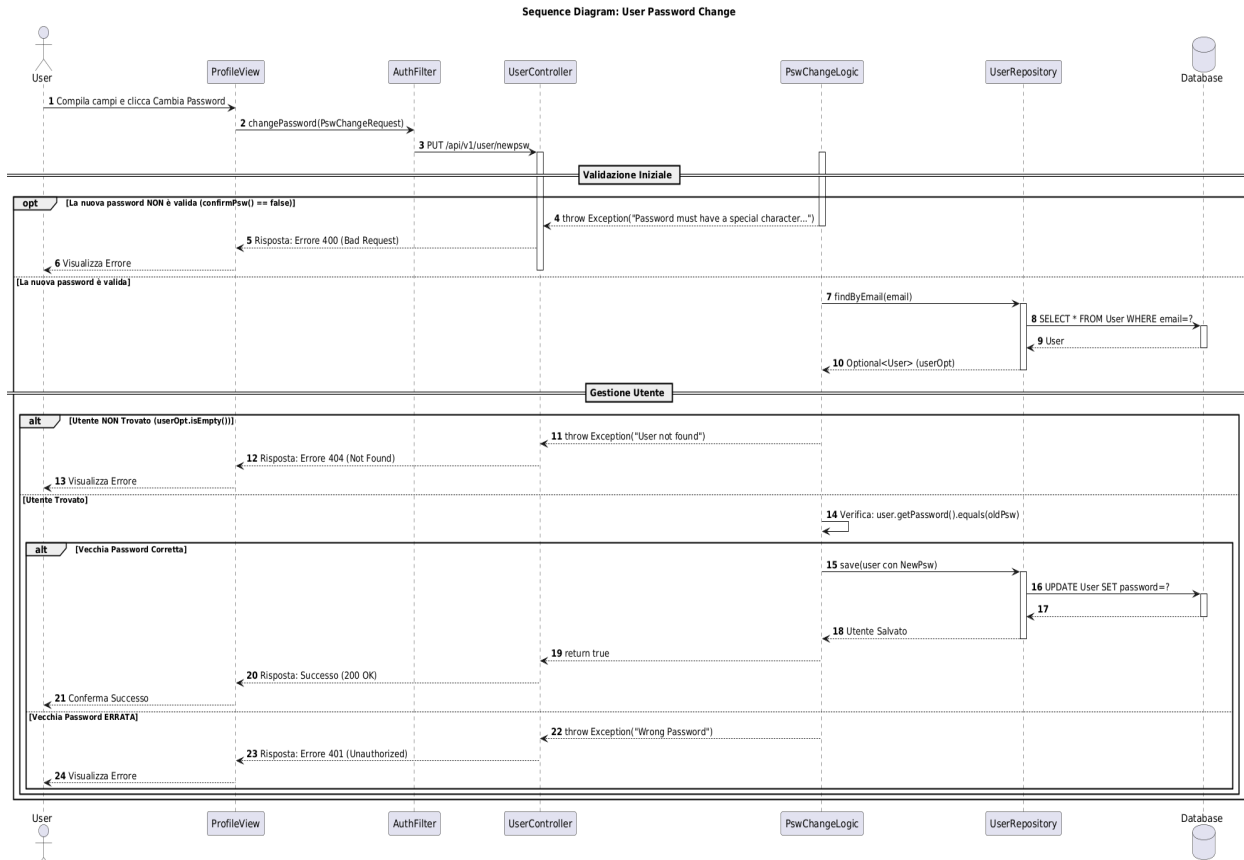


Figura 7 -Statechart “Cambio Password”

## 2.12.2.3 Ricerca Avanzata degli Immobili

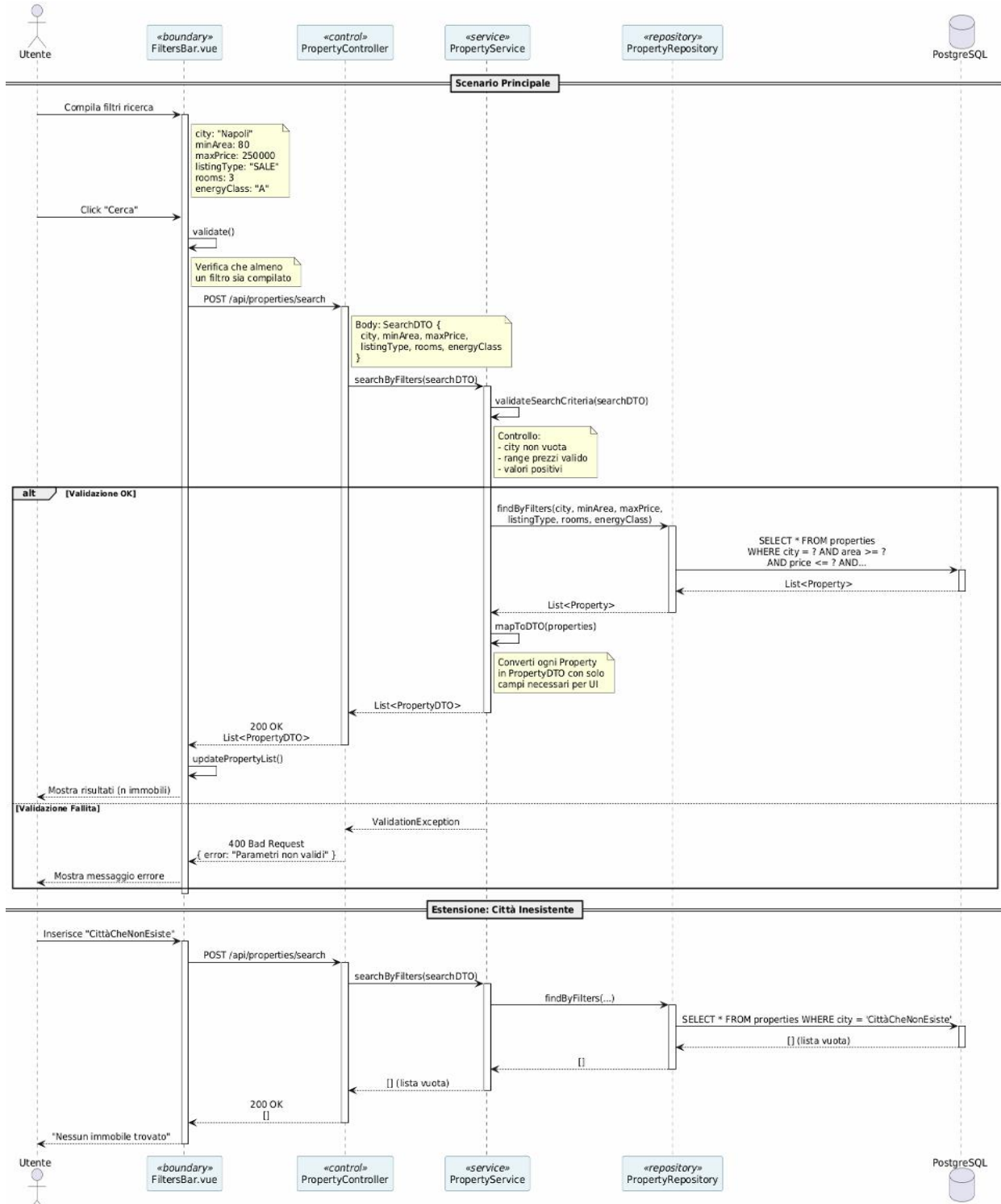


Figura 8 – Statechart “Ricerca Avanzata degli immobili”

## 2.12.2.4 Visualizza Statistiche

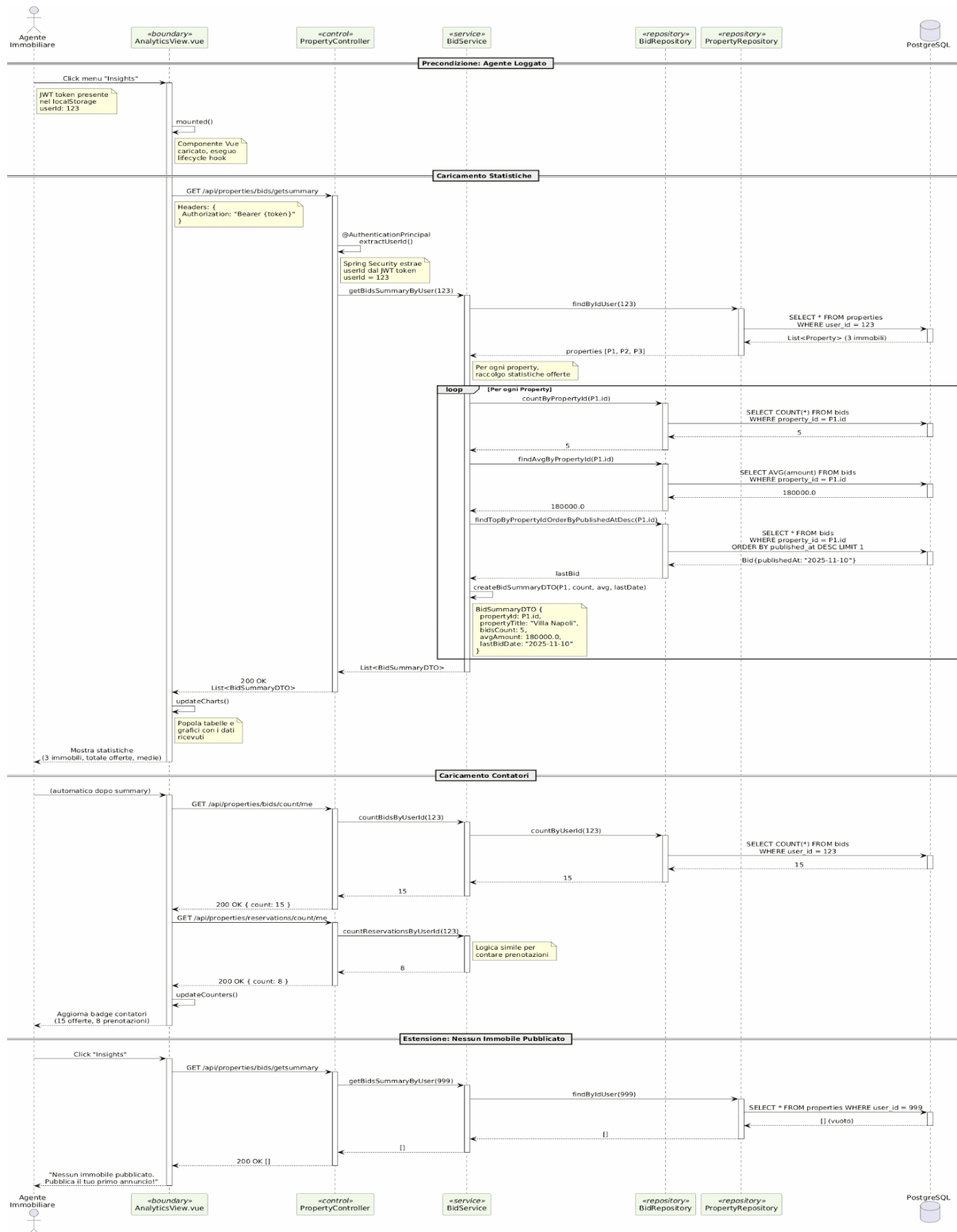


Figura 9 – Statechart "Visualizza Statistiche"

## 3 Documento di Design del Sistema

### 3.1 Introduzione

Il presente Documento di Design descrive l'architettura software e le scelte progettuali adottate per la realizzazione del sistema DietiEstates25. Tali decisioni sono basate su quanto analizzato nel Documento dei Requisiti Software.

### 3.2 Design Architeturale del Sistema

L'architettura del sistema DietiEstates25 è stata progettata per aderire pienamente ai requisiti di progetto, in particolare al vincolo fondamentale di un sistema distribuito moderno che prevede due macro-componenti indipendenti: un back-end e un front-end, per garantire la netta separazione tra l'interfaccia utente e la logica di business.

Il sistema si basa su un modello Client-Server evoluto in un'architettura Service-Based, che permette di gestire la complessità applicativa attraverso componenti modulari e indipendenti.

#### 3.2.1 Panoramica dei livelli

L'intera infrastruttura è suddivisa in tre livelli logici, ciascuno isolato e comunicante tramite interfacce standard:

1. **Presentation Layer (Frontend):** Sviluppato con il framework Vue.js, funge da Single Page Application (SPA). Questo strato è responsabile della gestione dello stato dell'interfaccia, della validazione dell'input lato client e della comunicazione asincrona con il backend tramite chiamate REST. È l'unico componente del sistema direttamente accessibile dall'utente finale.
2. **Application Logic Layer (Backend):** Rappresenta il cuore del sistema ed è composto da una suite di servizi Spring Boot (Java 17). Questo

livello non è un monolite; è invece segmentato in un Api-Gateway che funge da barriera e in molteplici servizi di dominio (User-Service, Property-Service) che eseguono i calcoli e le operazioni CRUD necessarie.

3. **Data Layer (Persistenza):** Il livello di persistenza è affidato a un database relazionale PostgreSQL, gestito esternamente tramite la piattaforma Supabase. Questa scelta permette di delegare la gestione, il backup e la disponibilità del dato a un servizio specializzato, garantendo al contempo alte prestazioni e conformità allo standard SQL.

### 3.2.2 Deployment e Containerizzazione

A differenza delle architetture tradizionali, DietiEstates25 adotta una strategia Cloud-Native:

- **Docker & Docker Compose:** Ogni componente del backend e del frontend è incapsulato in un container Docker, garantendo che l'ambiente di esecuzione sia identico dallo sviluppo alla produzione.

### 3.2.3 Architettura del Back-end

L'architettura del backend di DietiEstates25 è stata progettata per garantire modularità e indipendenza dei servizi, aderendo al paradigma dei sistemi distribuiti. La logica di business è stata isolata in servizi specializzati che comunicano tramite protocollo HTTP/REST, coordinati da un punto di accesso centralizzato.

### 3.2.3.1 Panoramica del Back-end Services Layer

Il backend adotta un approccio Service-Based volto a massimizzare la coesione interna e ridurre l'accoppiamento tra le diverse aree funzionali del dominio

Le scelte architetturali per il lato server sono state guidate dai seguenti principi fondamentali:

**1. Architettura Api-Gateway:** Per gestire la complessità derivante dalla distribuzione dei servizi, è stato implementato un Api-Gateway come unico punto di ingresso per il front-end. Questo componente si occupa di:

- Routing: Smistamento delle richieste in arrivo verso i servizi corretti.
- Astrazione: Nascondere la topologia di rete interna al client, esponendo un'unica interfaccia coerente.
- Centralizzazione: Gestione semplificata delle policy di sicurezza e dei flussi di traffico.

**2. Modularità dei Servizi (Java 17 & Spring Boot):** Il core del sistema è suddiviso in servizi indipendenti sviluppati con Java 17, sfruttando l'ecosistema Spring Boot per la gestione del ciclo di vita dei componenti:

- User Service: Responsabile della logica di gestione degli utenti, autenticazione e profili.
- Property Service: Gestisce l'intero dominio delle proprietà immobiliari, dalla pubblicazione alla ricerca.
- Ogni servizio è progettato per essere stateless, facilitando la scalabilità orizzontale all'interno dell'infrastruttura cloud.

**3. Containerizzazione e Orchestrazione:** Per garantire la portabilità tra l'ambiente di sviluppo locale e l'ambiente di produzione su Azure, l'intero backend è containerizzato tramite Docker.

- L'uso di docker-compose permette di orchestrare l'avvio sincronizzato dei servizi e del database.
- Ciascun container espone porte dedicate, permettendo una segregazione dei compiti a livello di rete virtuale.



### 3.2.2 Analisi Alternative prese in considerazione

**Microservizi:** L'architettura a microservizi presenta svariati vantaggi come isolamento delle varie componenti rendendo il sistema consistente in caso di guasti, ma anche rendere scalabile solo un componente che ne ha bisogno e permette di avere una suddivisione più isolata per ogni team del gruppo di lavoro. Non è stata scelta questa architettura in quanto ogni microservizio necessita di un proprio database e ogni chiamata tra servizi aggiunge dei millisecondi di latenza che sommati possono peggiorare l'esperienza utente. Quindi la complessità realizzativa è sicuramente elevata e potrebbe non garantire un usability corretta.

**Architettura Service-Based:** Questa architettura è quella che abbiamo implementato nella nostra soluzione. l'architettura service-based è più pragmatica. I servizi sono "macro", raggruppati per domini di business logici, e spesso condividono un unico database centrale.

I vantaggi sono molteplici come la semplicità di gestione essendo meno frammentato permette di monitorare e testare il codice in modo più diretto, inoltre avendo un solo database condiviso mantenere la coerenza dei dati è naturale.

Bisogna sicuramente tenere in conto che il fallimento di un singolo componente può interferire con il corretto funzionamento dell'app completa.

### 3.3 Dettaglio Componenti

#### 3.3.1 Stack Tecnologico Front-end

| Tecnologia        | Ruolo             | Motivazione                                                                                                                          |
|-------------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <b>Vue.js 3</b>   | Framework         | Offre un sistema di reattività efficiente e una struttura a componenti che facilita la manutenibilità.                               |
| <b>TypeScript</b> | Linguaggio        | Introdotta per garantire la tipizzazione statica, riducendo gli errori a runtime e migliorando l'integrazione con i DTO del backend. |
| <b>Pinia</b>      | State Management  | Gestisce in modo centralizzato e reattivo lo stato dell'autenticazione (token JWT) e i dati globali come i filtri di ricerca         |
| <b>Leaflet.js</b> | Mappe interattive | Libreria leggera per la gestione della cartografia interattiva, essenziale per la localizzazione degli immobili                      |
| <b>Vite</b>       | Build tool        | Per uno sviluppo rapido e build ottimizzate                                                                                          |
| <b>Axios</b>      | Networking        | Strumento utilizzato per effettuare le chiamate http verso il back-end                                                               |

#### 3.3.2 Stack Tecnologico Back-end

| Tecnologia           | Ruolo      | Motivazione                                                                                                                                       |
|----------------------|------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Spring Boot 3</b> | Framework  | Framework standard per la creazione di microservizi Java, che accelera lo sviluppo grazie alla configurazione automatica e all'ecosistema Spring. |
| <b>Java 17</b>       | Linguaggio | Introdotta per garantire la tipizzazione statica, riducendo gli errori a runtime e                                                                |

|                               |                    |                                                                                                                                |
|-------------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------|
|                               |                    | migliorando l'integrazione con i DTO del backend.                                                                              |
| <b><i>JWT RS256</i></b>       | Sicurezza          | Utilizza la crittografia asimmetrica per garantire un'autenticazione stateless e sicura, impedendo la manipolazione dei token. |
| <b><i>Spring Data JPA</i></b> | Persistenza        | Astrae l'accesso al database PostgreSQL tramite Hibernate, facilitando la gestione delle entità e delle relazioni.             |
| <b><i>PostgreSQL</i></b>      | Database           | Database relazionale robusto che garantisce l'integrità dei dati e supporta query complesse per la ricerca avanzata.           |
| <b><i>Docker</i></b>          | Containerizzazione | La containerizzazione garantisce la portabilità tra ambienti.                                                                  |

### 3.4 Design persistenza dei dati

A supporto di una corretta implementazione del database già nelle fasi iniziali della creazione del progetto, è stato creato uno schema concettuale che ha aiutato ad avere una prima bozza delle entità e le relazioni tra queste.

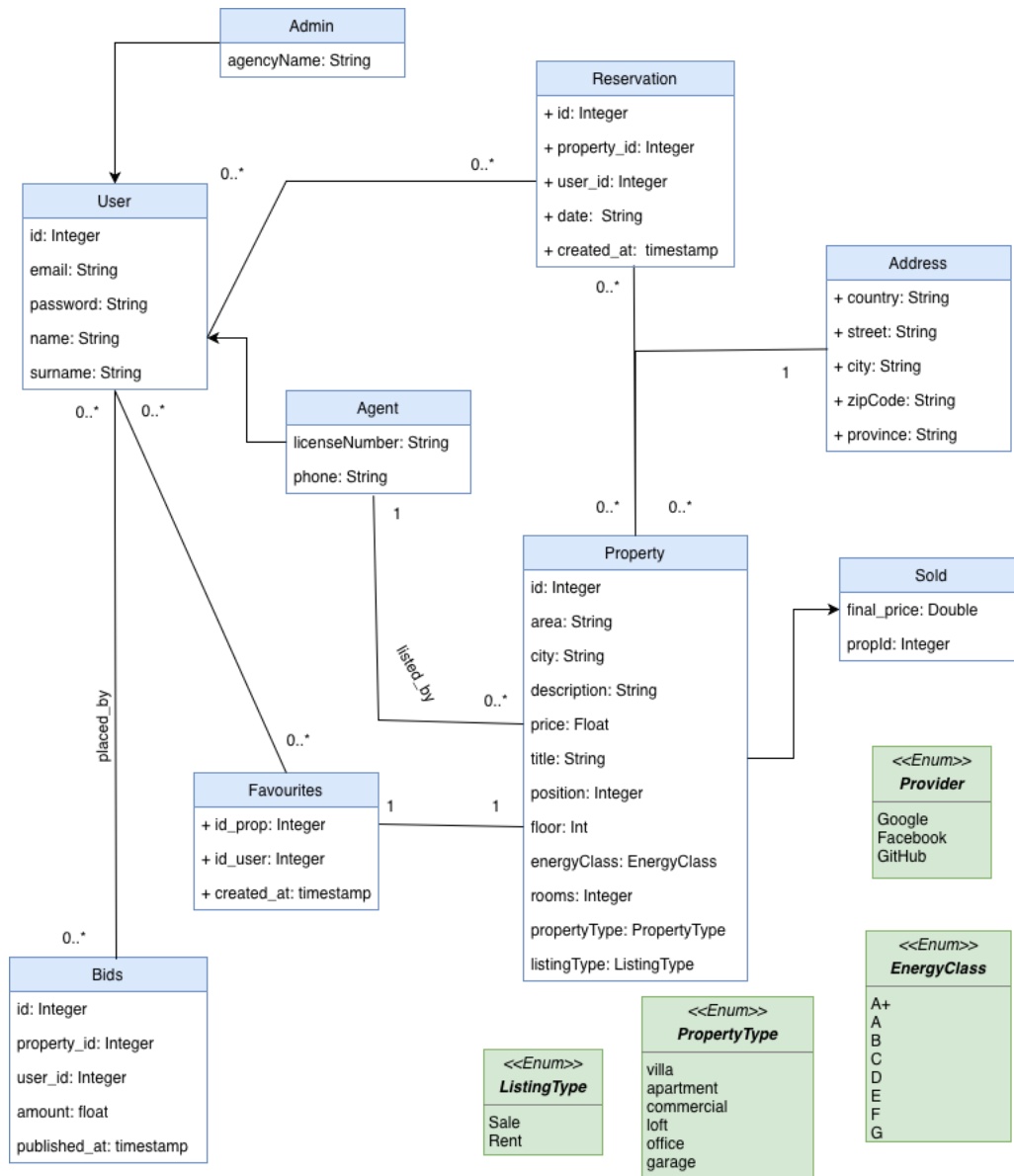


Figura 10 – Uml concettuale persistenza dei dati

Tale schema è stato infine ristrutturato e ridefinito per essere adattato all'utilizzo dell'ORM "Hibernate". Di seguito tutte le modifiche apportate:

- Le specializzazioni "Admin" e "Agent" sono state rimosse, inserendo un attributo all'interno della tabella "User" ovvero "role" che permette la distinzione tra i vari utenti. L'attributo phone è l'unico ad essere sopravvissuto alle modifiche e quindi è stato inglobato all'interno di "User".
- La classe "Address" è stata rimossa e tutti i campi sono stati spostati nella tabella "Property" come non nullable, inoltre sono stati aggiunti ulteriori campi per permettere la ricerca tramite mappa interattiva. Anche la classe "Sold" è stata rimossa in quanto non necessaria.
- Sono state corrette le molteplicità tra "Bids" e "User", "Favourite" e "User" e "Favourite" e "Property".

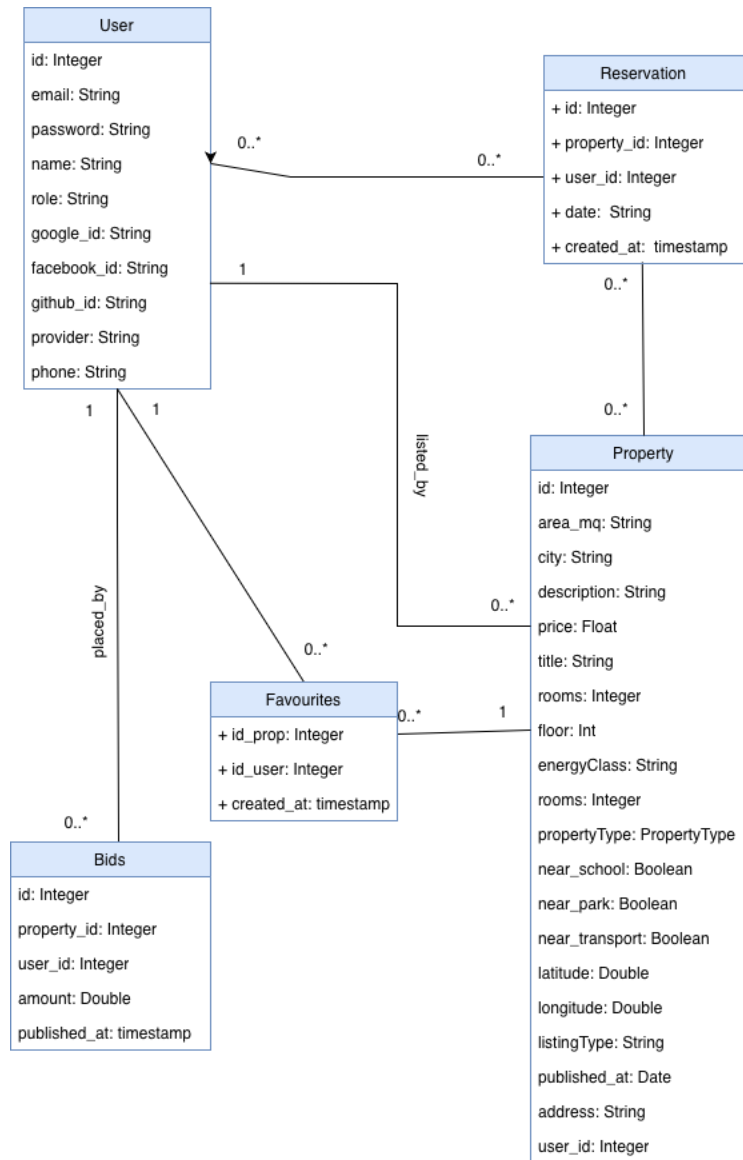


Figura 11 – Uml ristrutturato persistenza dei dati

## 3.5 Design della UI

Il design di DietiEstates25 è stato concepito come un'estensione funzionale dei requisiti di sistema, finalizzato a trasformare la complessità della ricerca immobiliare in un'esperienza d'uso fluida, intuitiva e tecnicamente rigorosa. L'interfaccia non è intesa come semplice elemento estetico, ma come un punto di contatto cognitivo volto a massimizzare l'efficienza degli attori coinvolti.

### 3.5.1 Paradigmi visive e architettura informativa

La morfologia visiva del sistema aderisce ai più moderni standard di *Material Design* e *Glassmorphism*, declinati per rispondere alle specifiche esigenze del mercato immobiliare digitale.

- **Dualismo Cromatico e Contrasto:** L'applicazione utilizza una palette basata su gradienti violacei e blu profondi, comunicando un senso di innovazione e solidità tecnologica. Il contrasto è ottimizzato per garantire la leggibilità: il testo bianco su sfondi scuri e l'uso del verde smeraldo per i prezzi e le etichette "Vendita" creano punti di ancoraggio visivo immediati, permettendo all'utente di identificare le informazioni chiave (prezzo e stato) in frazioni di secondo.
- **Gerarchia Tipografica e Semantica:** L'architettura informativa utilizza una scala tipografica chiara. I titoli delle proprietà (es. "Villa in centro") utilizzano un peso Bold per dominare lo spazio visivo, seguiti da sotto-intestazioni in grigio antracite per i dettagli tecnici (superficie, stanze). Questa gerarchia riduce il carico cognitivo, guidando l'occhio dall'elemento emozionale (la foto) a quello razionale (i dati tecnici).
- **Simbologia Iconografica:** L'uso di icone minimaliste (la lente per la ricerca, lo scudo per la sicurezza, il segnalibro per i preferiti) funge da linguaggio universale, permettendo una navigazione intuitiva che trascende la semplice lettura delle etichette testuali.

### 3.5.2 Euristiche di Usabilità e Affordance

Il design è stato validato seguendo le 10 euristiche di Nielsen, garantendo che l'utente abbia sempre il controllo del sistema.

- **Affordance Esplicita e Feedback:** Ogni elemento interattivo dichiara chiaramente la sua funzione. I pulsanti di azione ("Dettagli", "Cerca", "Pubblica") presentano angoli arrotondati e micro-interazioni (cambio di saturazione al passaggio del mouse) che confermano l'interattività. I messaggi di conferma, come la modale di "Conferma Logout", impediscono azioni involontarie chiedendo una validazione esplicita attraverso un design pulito e centrato. Inoltre, ogni operazione critica o a seguito di un input dell'utente riceve un toast come risposta indicando lo stato del sistema e se quell'azione ha avuto successo o meno.
- **Prevenzione dell'Errore nel Design dei Form:** Il processo di pubblicazione di un annuncio è suddiviso in step numerati (1-5). Questo approccio (stepper) evita l'overwhelming dell'utente. I campi di input utilizzano placeholder esplicativi (es. "Es. Napoli") e sistemi di validazione in tempo reale (come il tooltip "Please fill out this field"), impedendo l'invio di dati incompleti o errati.
- **Visualizzazione Analitica (Dashboard):** La dashboard per gli amministratori e gli agenti trasforma dati complessi in informazioni azionabili. L'uso di grafici a linee morbide per il "Trend Offerte" permette una comprensione immediata dell'andamento del mercato nel tempo, mentre le schede riassuntive in alto (Proprietà Totali, In Vendita, In



Affitto) forniscono un colpo d'occhio immediato sullo stato del catalogo.

### 3.5.3 Adattività e Performance dell'Interfaccia

L'interfaccia non è solo estetica, ma è progettata per essere uno strumento di lavoro reattivo e personalizzato.

- **Sincronia Geospaziale:** Il sistema integra la funzione "Ricerca su mappa", che permette di passare dalla visualizzazione a griglia a quella spaziale. Questa funzionalità risponde all'esigenza dell'utente di contestualizzare l'immobile nel tessuto urbano, collegando l'informazione testuale alla realtà geografica.
- **Accesso Ruolo-Dipendente:** L'interfaccia si adatta dinamicamente in base ai permessi dell'utente (Admin, Agente, Cliente). Ad esempio, l'utente "Admin" visualizza pulsanti di gestione ("Modifica", "Elimina") e badge di popolarità direttamente sulle card immobiliari, elementi che rimangono nascosti o semplificati per l'utente finale, garantendo pulizia visiva e sicurezza operativa.
- **Responsive Web Design:** Sebbene focalizzata sulla ricchezza della versione desktop per la gestione amministrativa, l'architettura dei componenti è basata su una griglia flessibile (Grid System), assicurando che le card degli immobili e i filtri di ricerca si riposizionino fluidamente su dispositivi tablet e mobile senza perdita di funzionalità.

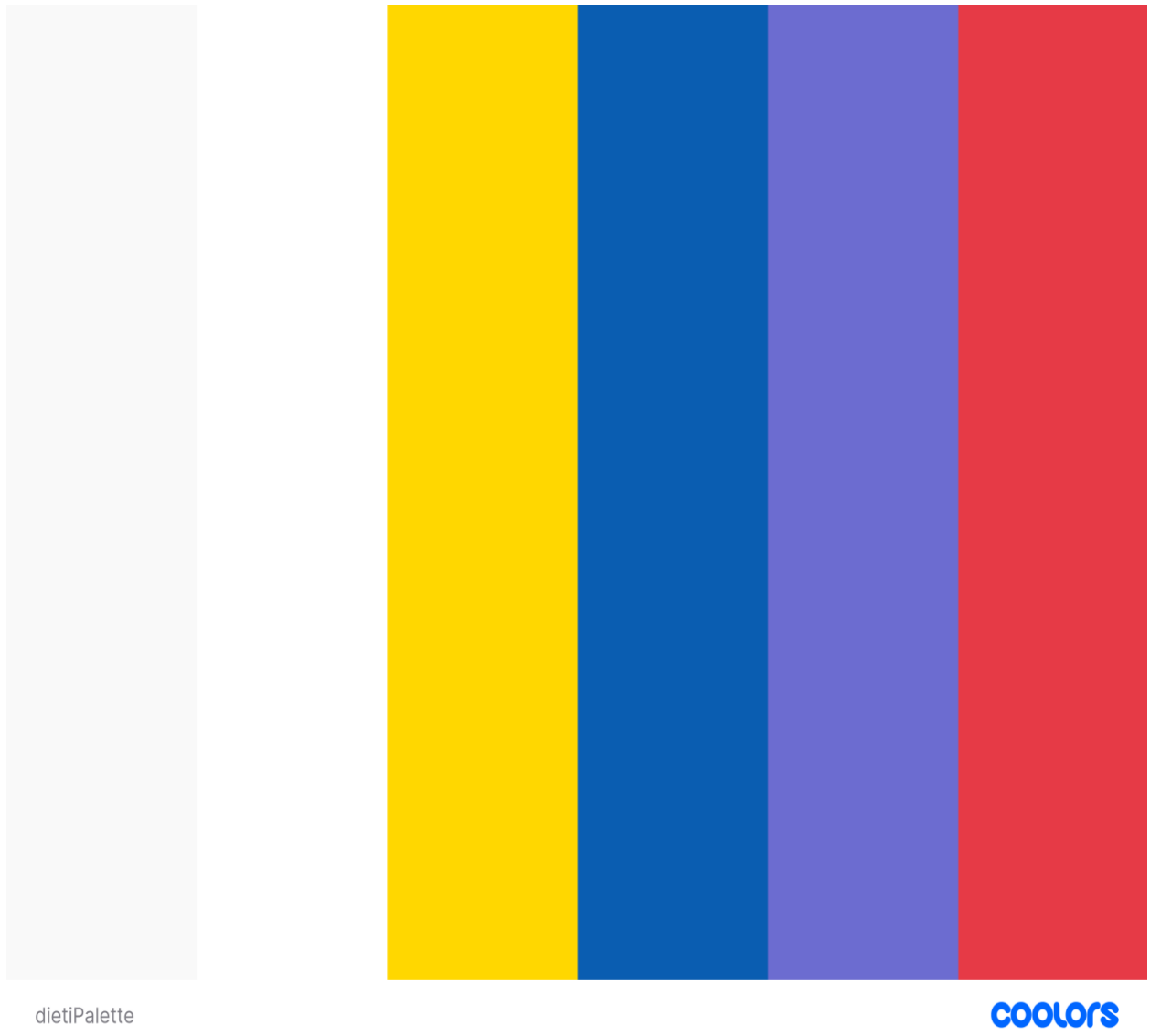
### 3.5.4 Coerenza e Standard

Il software mantiene una coerenza interna rigorosa. Gli elementi grafici (come il pulsante blu per le azioni primarie) e le spaziature (padding e margin) sono uniformi in tutte le pagine, dal profilo utente alla gestione degli annunci. Questa ripetizione di pattern crea "abitudine" nell'utente, riducendo drasticamente il tempo di apprendimento (learning curve) del sistema.

La palette dei colori scelta garantisce una leggibilità adeguata alle esigenze degli utenti, grazie a un attento bilanciamento tra colori primari, secondari e neutri. In particolare, l'utilizzo del blu come colore dominante per le azioni principali trasmette affidabilità e chiarezza, guidando l'utente nelle interazioni più rilevanti. Il giallo, impiegato come colore di accento, permette di evidenziare elementi importanti o stati di attenzione senza compromettere l'armonia visiva complessiva.

Le tonalità di grigio, invece, contribuiscono a creare una gerarchia visiva ben definita, distinguendo chiaramente tra contenuti primari e secondari e migliorando la scansione delle informazioni. Inoltre, il contrasto tra i colori è stato progettato per garantire accessibilità, facilitando la lettura anche in condizioni di utilizzo prolungato o su dispositivi diversi.

Infine, la coerenza nell'applicazione della palette cromatica in tutte le sezioni del software rafforza l'identità visiva del sistema e contribuisce a un'esperienza utente più intuitiva e riconoscibile.



*Figura 12 – Color Palette*

## Esempio contrasto del testo #1

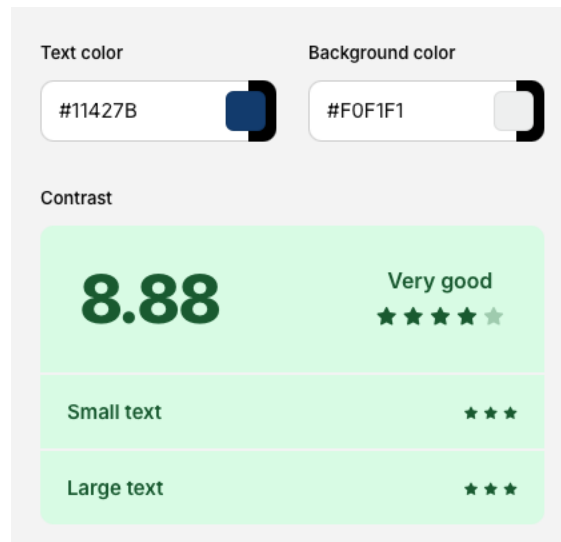


Figura 13 – Contrasto #1

## Esempio contrasto del testo #2

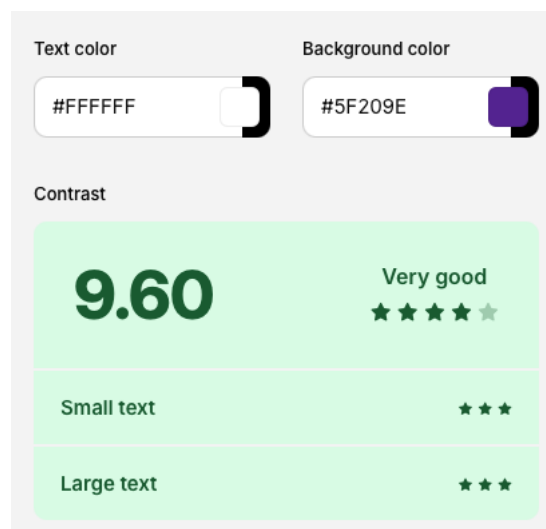
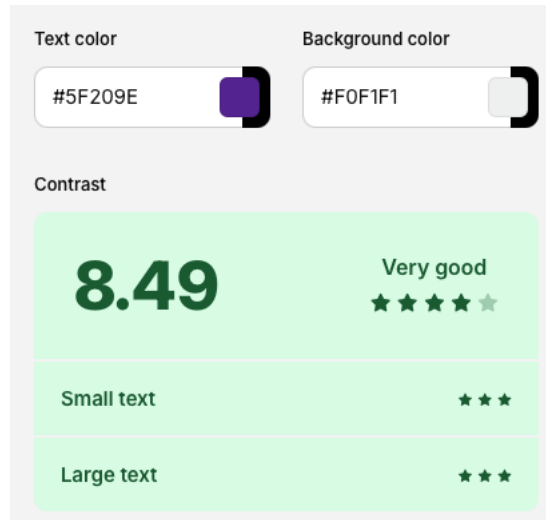


Figura 14 – Contrasto #2

## Esempio contrasto del testo #3

*Figura 15 – Contrasto #3*

### 3.6 Obiettivi di Qualità

Per guidare le scelte progettuali e architetture del sistema DietiEstates25, sono stati selezionati i seguenti attributi di qualità basati sullo standard ISO/IEC 25010. Ad ogni attributo è stato assegnato un valore di priorità da 1 (bassa) a 5 (critica).

| <i>Caratteristica</i> | <i>Priorità</i> | <i>Motivazione</i>                                                                                                                                                                                                                            |
|-----------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Sicurezza</b>      | <b>5</b>        | <i>Trattandosi di una piattaforma che gestisce dati personali e credenziali, la protezione contro accessi non autorizzati è il requisito primario.</i>                                                                                        |
| <b>Manutenibilità</b> | <b>5</b>        | <i>L'adozione di un'architettura Service-Based mira a isolare i domini (User, Property) per facilitare evoluzioni future senza regressioni.</i>                                                                                               |
| <b>Flexibility</b>    | <b>5</b>        | Priorità alta, correlata alla Manutenibilità. La traccia menziona possibili funzionalità future (es. "affitti brevi e case vacanze"). Il design modulare deve permettere di aggiungere nuove funzionalità senza ridisegnare l'intero sistema. |
| <b>Usabilità</b>      | <b>4</b>        | <i>Essenziale per garantire che utenti privati e agenti possano navigare tra le mappe e caricare annunci senza frizioni (UX).</i>                                                                                                             |

|                     |          |                                                                                                                 |
|---------------------|----------|-----------------------------------------------------------------------------------------------------------------|
| <b>Affidabilità</b> | <b>3</b> | <i>Un guasto in un servizio non critico (es. statistiche) non deve compromettere la ricerca degli immobili.</i> |
| <b>Efficienza</b>   | <b>3</b> | <i>Importante per le ricerche su mappa, ma subordinata alla consistenza dei dati e alla sicurezza.</i>          |

La definizione di queste priorità ha influenzato direttamente il design del sistema:

1. **Priorità Sicurezza:** Ha imposto l'uso di un API Gateway come unico punto di ingresso (Point of Enforcement) per la validazione dei token JWT e l'uso di chiavi asimmetriche (RS256) per garantire l'integrità delle comunicazioni tra i servizi oltre al conservare le chiavi crittografate.
2. **Priorità Manutenibilità:** Ha portato alla scelta dell'architettura Service-Based. Suddividendo la logica in macro-servizi (User-Service, Property-Service), riduciamo la complessità del codice rispetto a un monolito, facilitando il testing isolato e la manutenzione.
3. **Priorità Usabilità:** Ha richiesto l'integrazione di servizi esterni per automatizzare il reperimento di informazioni sui punti di interesse, riducendo il carico cognitivo dell'utente durante il caricamento di un annuncio.

## 3.7 Sicurezza

### 3.7.1 JWT Tokens

I token in questo sistema sono utilizzati per gestire l'autenticazione e l'autorizzazione in modo sicuro e stateless.

Il flusso è diviso principalmente tra il servizio “user-service, responsabile della generazione del token (alla registrazione o al login) e l’api-gateway. Nello specifico, al login dell’utente vengono generati 2 token: Access Token e Refresh Token

I Token sono firmati usando un algoritmo asimmetrico RS256, ovvero si ha una chiave privata nota solo al microservizio “user-service” che viene usata per firmare e una chiave pubblica conosciuta dal gateway ed utilizzata per validare.

Il Token contiene informazioni quali:

- ruolo (ADMIN , USER, AGENT)
- Id dell’utente
- data di scadenza (exp)

Il gateway ad ogni richiesta intercettata effettua la validazione del token, preleva i dati conservati e inoltra la richiesta al servizio corretto.

#### 3.7.1.1 Motivazioni della scelta

Abbiamo scelto questo modello che utilizza la crittografia asimmetrica in quanto risulta essere il più sicuro e difficile da rompere.

Integrità: Garantisce integrità in quanto qualsiasi modifica al contenuto del token invaliderebbe immediatamente la firma, facendo in modo che il server rifiuti le successive richieste.



Stateless: Non richiede di salvare lo stato della sessione sul database rendendo il sistema più scalabile e meno vulnerabile ad attacchi di session fixation.

La fase di Issue (generazione) è delegata esclusivamente allo user-service, il quale gestisce la logica di business relativa a registrazione e login. Al termine di un'autenticazione corretta, il servizio genera una coppia di token (Access e Refresh) firmati con algoritmo asimmetrico RS256.

La fase di Validation è invece centralizzata nell'API Gateway. Questo componente intercetta ogni richiesta HTTP entrante, verifica l'integrità della firma tramite la chiave pubblica e, in caso di esito positivo, instrada la richiesta verso i servizi di back-end. Tale approccio permette ai singoli microservizi di operare in modo disaccoppiato: essi possono processare le richieste confidando nella validazione già effettuata a monte dal gateway, garantendo così una maggiore efficienza e una separazione netta delle responsabilità (Separation of Concerns).

Scadenza: limitando il tempo di vita di un token si riduce la finestra di rischio se questo dovesse essere rubato.

## 3.8 Codice Sorgente sviluppato

Al fine di garantire la totale riproducibilità dell'ambiente software e ottimizzare la collaborazione all'interno del team, abbiamo adottato una strategia di containerizzazione basata su Docker. Di seguito sono riportati i file di configurazione e gli script di automazione che costituiscono l'infrastruttura di build e deploy del progetto.

### 3.8.1 Configurazione Docker e Script di Deploy

```
services:
  user-service:
    build: ./backend/user-service
    container_name: user-service
    ports:
      - "8080:8080"
    env_file:
      - ./backend/user-service/.env
    networks:
      - dieti-network

  property-service:
    build: ./backend/property-service
    container_name: property-service
    ports:
      - "8081:8081"
    env_file:
      - ./backend/property-service/.env
    networks:
      - dieti-network
    volumes:
      - ./uploads:/app/uploads

  api-gateway:
```

```
build: ./backend/api-gateway
container_name: api-gateway
ports:
  - "8082:8082"
env_file:
  - ./backend/api-gateway/.env
depends_on:
  - user-service
  - property-service
networks:
  - dieti-network

frontend:
build:
  context: ./frontend
  target: dev-stage
container_name: frontend
ports:
  - "5173:5173"
environment:
  - VITE_API_URL=http://localhost:8082
  - VITE_API_USER_URL=/user
  - VITE_API_PROPERTY_URL=/properties
depends_on:
  - api-gateway
networks:
  - dieti-network

networks:
  dieti-network:
    driver: bridge
```

*Codice 1 – docker-compose.yml*

### 3.8.2 Dockerfile Front-end

```
# build stage
FROM node:22-alpine as build-stage
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build-only

# development stage
FROM node:22-alpine as dev-stage
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 5173
CMD ["npm", "run", "dev", "--", "--host"]

# production stage
FROM nginx:stable-alpine as production-stage
COPY --from=build-stage /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Codice 3 – frontend /Dockerfile

### 3.8.3 Dockerfile User-Service

```
FROM eclipse-temurin:17-jdk
VOLUME /tmp
COPY target/*.jar app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

Codice 4 – user-service/Dockerfile

### 3.8.4 Dockerfile Property-Service

```
FROM eclipse-temurin:17-jdk
VOLUME /tmp
COPY target/*.jar app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

Codice 5 – property-service/Dockerfile

## 3.9 Analisi delle Scelte Configurative e di Deploy

La strategia di deployment di DietiEstates25 è imperniata sull'uso di Docker, una scelta tecnologica che garantisce la totale riproducibilità dell'ambiente software e l'isolamento dei processi. L'architettura dei container è stata meticolosamente progettata per coniugare modularità, sicurezza e leggerezza delle immagini, eliminando le tipiche criticità legate alle divergenze tra gli ambienti di sviluppo, testing e produzione.

### Front-end Multi-Stage Build:

- Stage 1 (Build): Usa un'immagine Node.js pesante per compilare il codice Vue/TypeScript.
- Stage 2 (Production): Usa un'immagine Nginx per servire solo i file statici compilati. Questo riduce drasticamente i tempi di download e avvio del container.

**Back-end (Spring Boot):** I Dockerfile dei servizi user e property utilizzano immagini Maven per la compilazione, per poi eseguire il binario JAR su una JRE (Java Runtime Environment) ottimizzata. Questa struttura garantisce che l'ambiente di esecuzione sia privo di strumenti di build non necessari, aumentando la sicurezza e la portabilità del sistema

L'intero ecosistema è orchestrato tramite Docker Compose, che funge da supervisore per il ciclo di vita dei servizi, definendo reti virtuali isolate e volumi di persistenza.

- **Isolamento e Configurazione:** Ogni servizio (User, Property, Gateway) opera in un ambiente isolato, comunicando esclusivamente attraverso la rete interna definita dall'orchestratore.
- **Gestione della Sicurezza:** In conformità con le best practices, le chiavi crittografiche per la validazione dei JWT e le credenziali di accesso al database PostgreSQL su Supabase non sono incorporate nelle immagini. Esse vengono iniettate a runtime tramite volumi o variabili d'ambiente configurate nel docker-compose.yml, garantendo la massima flessibilità e protezione dei dati sensibili.

## 4 Testing

Per la realizzazione della batteria di test unitari automatici del backend, abbiamo adottato il framework **JUnit 5**, integrato con **Mockito** e **Spring Test (MockMvc)**.

JUnit 5 rappresenta lo standard per il testing in ambiente Java e si basa sui principi architetturali tipici dei framework **xUnit**, garantendo modularità, leggibilità e automazione nell'esecuzione dei test.

Le principali caratteristiche del framework utilizzato sono:

- **Struttura dei test:** organizzazione in classi di test contenenti metodi annotati con `@Test`, analogamente alla struttura classica dei framework *xUnit*.

- **Assertzioni:** utilizzo delle asserzioni fornite da JUnit e delle verifiche HTTP di MockMvc (`status().isOk()`, `status().isBadRequest()`) per controllare i risultati attesi.
- **Mocking delle dipendenze:** utilizzo di Mockito (`@MockBean`) per isolare il controller dalle dipendenze esterne (service layer), permettendo di testare esclusivamente il comportamento dell'unità in esame.
- **Simulazione richieste HTTP:** utilizzo di MockMvc per simulare chiamate REST verso i controller senza avviare un server reale, garantendo test rapidi e indipendenti dall'infrastruttura.

Questa combinazione di strumenti consente di realizzare test isolati, deterministici e ripetibili, riducendo la dipendenza da componenti esterni come database o sistemi di autenticazione.

## 4.1 Strategia Utilizzata

Per il testing dei controller REST abbiamo adottato un approccio Black Box mentre per i service un approccio White-box .

### 4.1.1 Test Black-Box

Nei test Black-box, i test verificano:

- gli input ricevuti tramite richiesta HTTP (parametri di query)
- l'output prodotto (codice di stato HTTP e risposta)

senza effettuare assunzioni sull'implementazione interna del metodo o sulla logica del service sottostante.

Questo approccio risulta particolarmente adatto al testing dei controller REST, in quanto consente di validare il comportamento dell'API dal punto di vista dell'utente o del client che effettua la chiamata.

### Strategia N-WECT (Normal Weak Equivalence Class Testing)

Per la definizione dei casi di test abbiamo adottato la tecnica **N-WECT (Normal Weak Equivalence Class Testing)**.

L'obiettivo è suddividere il dominio di input in classi di equivalenza, distinguendo tra:

- classi valide
- classi non valide

Nel caso del metodo `searchByBounds(lat, lon, radiusKm)` sono state individuate tre classi di equivalenza per ciascun parametro:

- **latitudine (lat):** valida se compresa tra -90 e 90
- **longitudine (lon):** valida se compresa tra -180 e 180
- **raggio (radiusKm):** valida se compreso tra 0 e 40075

Applicando la formula dell'N-WECT:

Numero test case = Max(numero classi per parametro)  
si ottiene:

$\text{Max}(3, 3, 3) = 3$  test case

Sono quindi stati progettati tre casi di test:

1. Un caso con latitudine non valida
2. Un caso con longitudine non valida
3. Un caso con tutti i parametri validi

Questa scelta garantisce una copertura significativa del dominio di input mantenendo la suite di test compatta ed efficiente. Di sotto è mostrato come abbiamo visualizzato le adatte CE per ogni parametro (classi di equivalenza aderenti all'approccio Black-box).

**(<sup>1</sup>/<sub>4</sub>)searchByBounds(Double lat, Double lon, Double radiusKm)**

-double lat



| Classe di equivalenza | Intervallo     | Validità   |
|-----------------------|----------------|------------|
| CE1                   | lat < -90      | Non valida |
| CE2                   | lat > 90       | Non valida |
| CE3                   | -90 ≤ lat ≤ 90 | Valida     |

-double lon

| Classe di equivalenza | Intervallo       | Validità   |
|-----------------------|------------------|------------|
| CE1                   | lon < -180       | Non valida |
| CE2                   | lon > 180        | Non valida |
| CE3                   | -180 ≤ lon ≤ 180 | Valida     |

-double radiuskm

| Classe di equivalenza | Intervallo              | Validità   |
|-----------------------|-------------------------|------------|
| CE1                   | radiuskm < 0            | Non valida |
| CE2                   | radiuskm > 40.075       | Non valida |
| CE3                   | 0.1 ≤ radiuskm ≤ 40.075 | Valida     |

```
@WebMvcTest(SearchController.class)
@AutoConfigureMockMvc(addFilters = false)
class SearchControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private SearchService searchService;

    @MockBean
    private JwtUtil jwtUtil;
```

```
@MockBean
private JwtAuthenticationFilter jwtAuthenticationFilter;
```

```
@Test
```

```
@DisplayName("TC1 - lat < -90 → 400")
```

```
void tc1_invalidLat() throws Exception {
```

```
    System.out.println("\nTC1 - LAT INVALIDA");
```

```
    mockMvc.perform(get("/search/bybounds")
```

```
        .param("lat", "-100")
```

```
        .param("lon", "10")
```

```
        .param("radiusKm", "10"))
```

```
        .andExpect(status().isBadRequest());
```

```
    System.out.println("✓ TC1 PASS");
```

```
}
```

```
@Test
```

```
@DisplayName("TC2 - lon > 180 → 400")
```

```
void tc2_invalidLon() throws Exception {
```

```
    System.out.println("\nTC2 - LON INVALIDA");
```

```
    mockMvc.perform(get("/search/bybounds")
```

```
        .param("lat", "45")
```

```
        .param("lon", "200")
```

```
        .param("radiusKm", "10"))
```

```
        .andExpect(status().isBadRequest());
```

```
    System.out.println("✓ TC2 PASS");
```

```
}
```

```
@Test
```

```
@DisplayName("TC3 - Parametri validi → 200")
```

```
void tc3_validInput() throws Exception {
```

```

System.out.println("\nTC3 - PARAMETRI VALIDI");

when(searchService.searchByBounds(45.0, 10.0, 10.0))
    .thenReturn(List.of());

mockMvc.perform(get("/search/bybounds")
    .param("lat", "45")
    .param("lon", "10")
    .param("radiusKm", "10"))
    .andExpect(status().isOk());

System.out.println("✓ TC3 PASS");
}
}

```

*Codice 6 – searchByBounds tests*

## (2/4) **userLogin(LoginRequest req)**

L'oggetto LoginRequest contiene :

- email
- password

Restituisce:

- **200 OK** con LoginResponse se le credenziali sono valide
- **401 Unauthorized** se le credenziali sono errate
- **500 Internal Server Error** in caso di errore inatteso

Anche qui abbiamo utilizzato un approccio **Black Box**, considerando esclusivamente:

- input forniti nel body JSON
- codice di stato HTTP restituito

## Identificazione delle Classi di Equivalenza

L'analisi del dominio di input è stata effettuata secondo approccio **Functionality Based**, considerando i vincoli funzionali applicati ai campi email e password.

### Classi di equivalenza

String password

| Classe di equivalenza | Intervallo                  | Validità   |
|-----------------------|-----------------------------|------------|
| CE1                   | Password vuota/non corretta | Non valida |
| CE2                   | Password !=empty/corretta   | Valida     |

String email

| Classe di equivalenza | Intervallo                              | Validità   |
|-----------------------|-----------------------------------------|------------|
| CE1                   | Email sintatticamente valida            | Valida     |
| CE2                   | Email sintatticamente non valida(not @) | Non valida |

### Applicazione della N-WECT

- Password → 2 classi
- Email → 2 classi

Applicando la formula:

Numero test case = Max(numero classi per parametro)  
si ottiene:

Max(2, 2) = 2 test case

```
@WebMvcTest(controllers = AuthController.class)
@AutoConfigureMockMvc(addFilters = false)
class AuthControllerTest {

    @Autowired
```

```

private MockMvc mockMvc;

@MockBean
private AuthService authService;

@MockBean
private org.springframework.security.authentication.AuthenticationManager authenticationManager;

@MockBean
private org.springframework.security.core.userdetails.UserDetailsService userDetailsService;

@MockBean
private HeaderAuthenticationFilter headerAuthenticationFilter;

// =====
// TC1 - EMAIL NON VALIDA
// =====

@Test
@DisplayName("TC1 - Email non valida → 400")
void tc1_invalidEmail() throws Exception {

    String json = """
        {
            "email": "email_non_valida",
            "password": "Password1@"
        }
        """,

    System.out.println("\n=====");
    System.out.println("AVVIO TC1 - EMAIL NON VALIDA");
    System.out.println("JSON INVIATO:");
    System.out.println(json);

    MvcResult result = mockMvc.perform(post("/auth/login")
        .contentType(MediaType.APPLICATION_JSON)
        .content(json))
        .andDo(print())

```

```

        .andExpect(status().isBadRequest())
        .andReturn();

        System.out.println("STATUS RICEVUTO: " + result.getResponse().getStatus());
        System.out.println("BODY RISPOSTA:");
        System.out.println(result.getResponse().getContentAsString());

        System.out.println("TC1 SUPERATO ");
        System.out.println("=====\\n");
    }

    // =====
    // TC2 - CREDENZIALI VALIDE MA UTENTE NON TROVATO
    // =====

    @Test
    @DisplayName("TC2 - Credenziali valide ma utente non trovato → 401")
    void tc3_validButUnauthorized() throws Exception {

        when(authService.userLogin(any(), any(), any()))
            .thenReturn(Optional.empty());

        String json = ""
            {
                "email": "test@mail.com",
                "password": "Password1@"
            }
            """,

        System.out.println("\\n=====");
        System.out.println("AVVIO TC2 - UTENTE NON TROVATO");
        System.out.println("JSON INVIATO:");
        System.out.println(json);

        MvcResult result = mockMvc.perform(post("/auth/login")
            .contentType(MediaType.APPLICATION_JSON)
            .content(json))

```

```

        .andDo(print())
        .andExpect(status().isUnauthorized())
        .andReturn();

    System.out.println("STATUS RICEVUTO: " + result.getResponse().getStatus());
    System.out.println("BODY RISPOSTA:");
    System.out.println(result.getResponse().getContentAsString());

    System.out.println("TC2 SUPERATO ");
    System.out.println("=====\\n");
}
}

```

*Codice 7 – userLogin() tests*

### 4.1.2 Testing White-box

Per il testing white box abbiamo pensato ad un approccio service-based

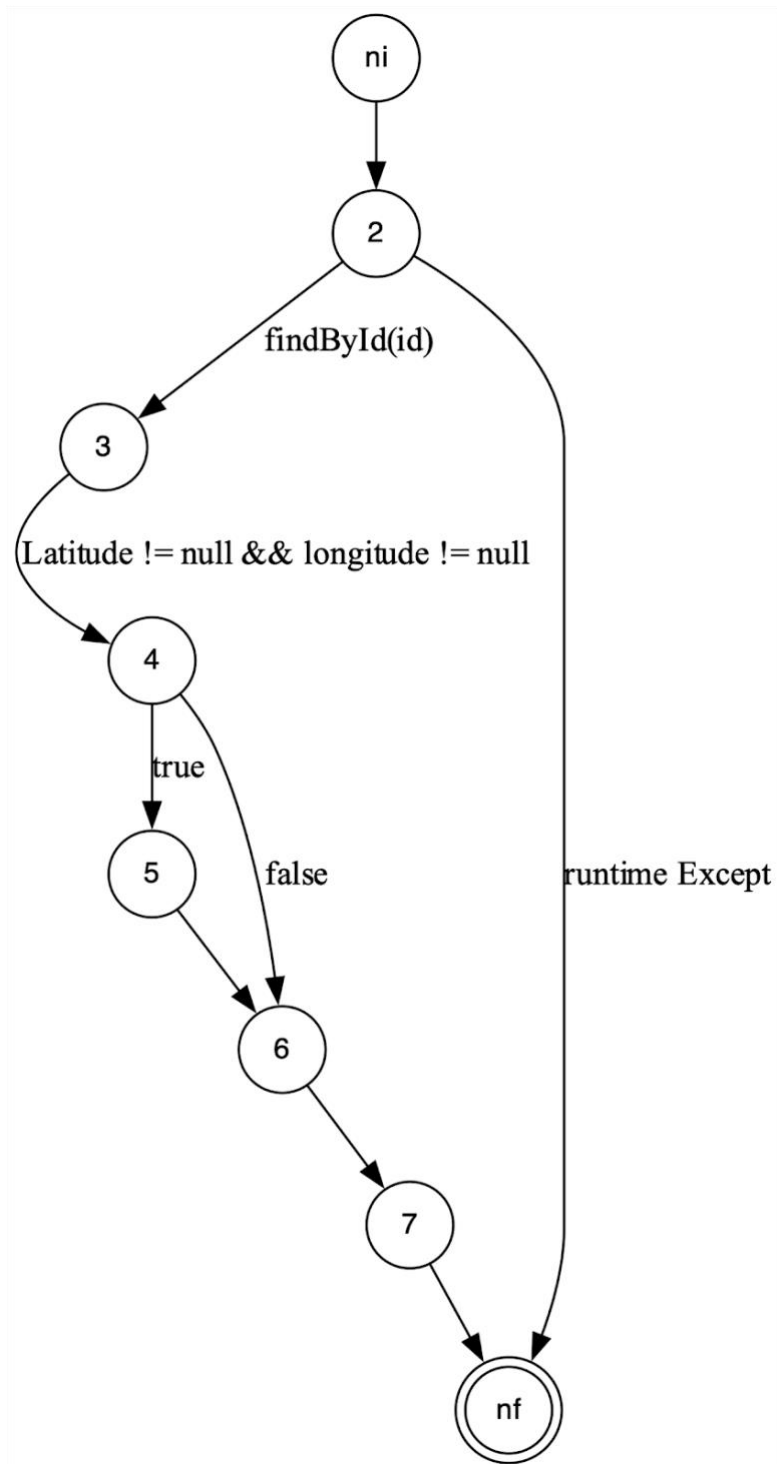
**(3/4) public PropertyUpdatedDTO updateProperty(Long id,  
PropertyUpdatedDTO updateDTO)**

Codice analizzato:

```
(1) public PropertyUpdateDTO updateProperty(Long id, PropertyUpdateDTO updateDTO) {  
    (2) Property property = propertyRepository.findById(id)  
        .orElseThrow(() -> new RuntimeException("Property not found with id: " + id));  
  
    (3) property.setTitle(updateDTO.getTitle());...  
  
    (4) if (updateDTO.getLatitude() != null && updateDTO.getLongitude() != null) {  
  
        (5) property.setLatitude(updateDTO.getLatitude());  
  
    }  
  
    (6) Property updatedProperty = propertyRepository.save(property);  
  
    (7) return new PropertyUpdateDTO(  
        updatedProperty.getTitle()...  
    }  
}
```

*Codice 8 – updateProperty()*



*Control flow graph**Figura 16 – Flow Graph “updateProperty”*

*Cammino P1*

Property non trovata:

$1 \rightarrow 2 \rightarrow \mathbf{nf}$

*Cammino P2*

Property trovata, coordinate assenti:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow \mathbf{nf}$

*Cammino P3*

Property trovata, coordinate presenti:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow \mathbf{nf}$

**Numero di test per ottenere le TER al 100%: 3**

| <b>Caso di test</b> | <b>Oracolo</b> | <b>Cammino coperto</b>  |
|---------------------|----------------|-------------------------|
| TC1                 | Error          | 1, 2, nf                |
| TC2                 | Success        | 1,2,3, 4, 6, 7, nf      |
| TC3                 | Error          | 1, 2, 3, 4, 5, 6, 7, nf |

**(4/4) public void addFavouriteProperty(Long userId, Long propId)**

Codice analizzato:

```
public void addFavouriteProperty(Long userId, Long propId) {  
    (1) var id = new FavouriteId(userId, propId);  
    (2) if (favouriteRepository.existsById(id))  
        return;  
    (3) try {  
    (4)     favouriteRepository.save(new Favourite(id));  
        } catch (Exception e) {  
        }  
    (nf) }
```

*Codice 9 – addFavouriteProperty()*

Control flow graph:

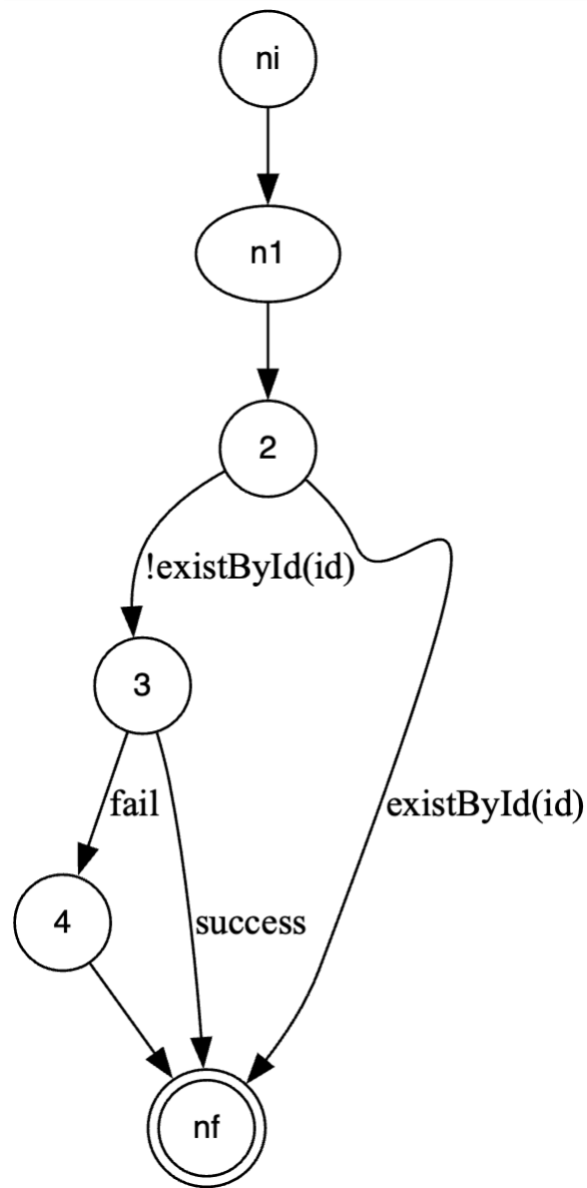


Figura 17 – Control flow graph “addFavourite”

```

@ExtendWith(MockitoExtension.class)
class FavouriteServiceTest {

    @Mock
    private FavouriteRepository favouriteRepository;

    @InjectMocks
    private FavouriteService favouriteService;

    @Test
    void testAddFavourite_PropertyAlreadyFavourite_ShouldReturnEarly() {
        Long userId = 1L;
        Long propId = 34L;
        var id = new FavouriteId(userId, propId);

        when(favouriteRepository.existsById(id)).thenReturn(true);
        favouriteService.addFavouriteProperty(userId, propId);

        verify(favouriteRepository, times(1)).existsById(id);
        verify(favouriteRepository, never()).save(any());
    }

    @Test
    void testAddFavourite_PropertyNotFavourite_ShouldSaveSuccessfully() {
        Long userId = 1L;
        Long propId = 12L;
        var id = new FavouriteId(userId, propId);

        when(favouriteRepository.existsById(id)).thenReturn(false);
        favouriteService.addFavouriteProperty(userId, propId);

        verify(favouriteRepository, times(1)).existsById(id);
        verify(favouriteRepository, times(1)).save(any(Favourite.class));
    }
}

```

```

@Test
void testAddFavourite_WhenSaveThrowsException_ShouldHandleGracefully() {

    Long userId = 1L;
    Long propId = 99L;
    var id = new FavouriteId(userId, propId);

    when(favouriteRepository.existsById(id)).thenReturn(false);
    doThrow(new RuntimeException("Database Error")).when(favouriteRepository).save(any(Favourite.class));

    assertDoesNotThrow(() -> {
        favouriteService.addFavouriteProperty(userId, propId);
    }, "Il servizio deve gestire l'eccezione internamente per garantire l'idempotenza");

    verify(favouriteRepository, times(1)).save(any(Favourite.class));
}

@Test
void testRemoveFavourite_Success() {
    Long userId = 10L;
    Long propId = 50L;

    favouriteService.removeFavourite(userId, propId);

    verify(favouriteRepository, times(1)).deleteByIdUserIdAndIdPropertyId(userId, propId);
}
}

```

*Codice 10 – addFavourite Test*

**Numero di test per ottenere le TER al 100%: 3**

| Caso di test | Oracolo | Cammino coperto |
|--------------|---------|-----------------|
| TC1          | Error   | 1, 2, nf        |
| TC2          | Success | 1,2,3, nf       |
| TC3          | Error   | 1,2,3,4,nf      |

## 5 Versioning e Qualità del codice

Il versioning rappresenta un elemento fondamentale nello sviluppo di un progetto software, in quanto consente di tracciare in modo sistematico tutte le modifiche apportate al codice nel tempo. Attraverso l'utilizzo di sistemi di controllo di versione come Git, è possibile gestire in maniera efficace il lavoro collaborativo tra più sviluppatori, mantenere una cronologia completa delle modifiche, facilitare il ripristino di versioni precedenti e ridurre il rischio di errori o conflitti. Inoltre, il versioning permette di analizzare il contributo dei singoli membri del team, migliorare l'organizzazione del lavoro e garantire una maggiore qualità e tracciabilità del prodotto finale.

Al seguente link è disponibile il codice sorgente dell'applicazione prodotta:  
[https://github.com/AgnelloSant/DietiEstates25\\_demeo\\_carrese](https://github.com/AgnelloSant/DietiEstates25_demeo_carrese)

L'attività di sviluppo è stata analizzata considerando l'intero repository GitHub, includendo i principali branch utilizzati durante il progetto. In particolare, è stato utilizzato lo strumento "Contributors" per analizzare i contributi dei membri del team .

## Contributors

Period: All

Contributions: Commits

Contributions per week to FinalBranch, excluding merge commits



Figura 18 – Contributors



## Contributors

Contributions per week to main, excluding merge commits

Period: All ▾

Contributions: Commits ▾



Figura 19 – Commits Over Time “main”

## Contributors

Period: All ▾

Contributions: Commits ▾

Contributions per week to DesignConsistencyUp, excluding merge commits



Figura 20 – Commits Over Time “DesignConsistencyUp”

Contributors

Contributions per week to AnalyticsUpdate, excluding merge commits



Figura 21 – Commits Over Time “AnaliticsUpdate”

## 5.2 Report Qualità del codice

Il presente report ha l'obiettivo di valutare la qualità del codice del progetto attraverso l'utilizzo di SonarQube, una piattaforma ampiamente diffusa per l'analisi statica del software. Questo strumento consente di esaminare il codice sorgente in modo automatico, individuando potenziali criticità legate a bug, vulnerabilità di sicurezza, code smell e problemi di manutenibilità.

L'analisi effettuata permette di ottenere una visione complessiva dello stato qualitativo del progetto, evidenziando le aree che necessitano di miglioramenti e supportando il team nello sviluppo di codice più affidabile, sicuro e facilmente mantenibile. In particolare, i risultati forniti da SonarQube consentono di monitorare metriche fondamentali e di adottare buone pratiche di sviluppo, contribuendo al miglioramento continuo del prodotto software lungo tutto il suo ciclo di vita.

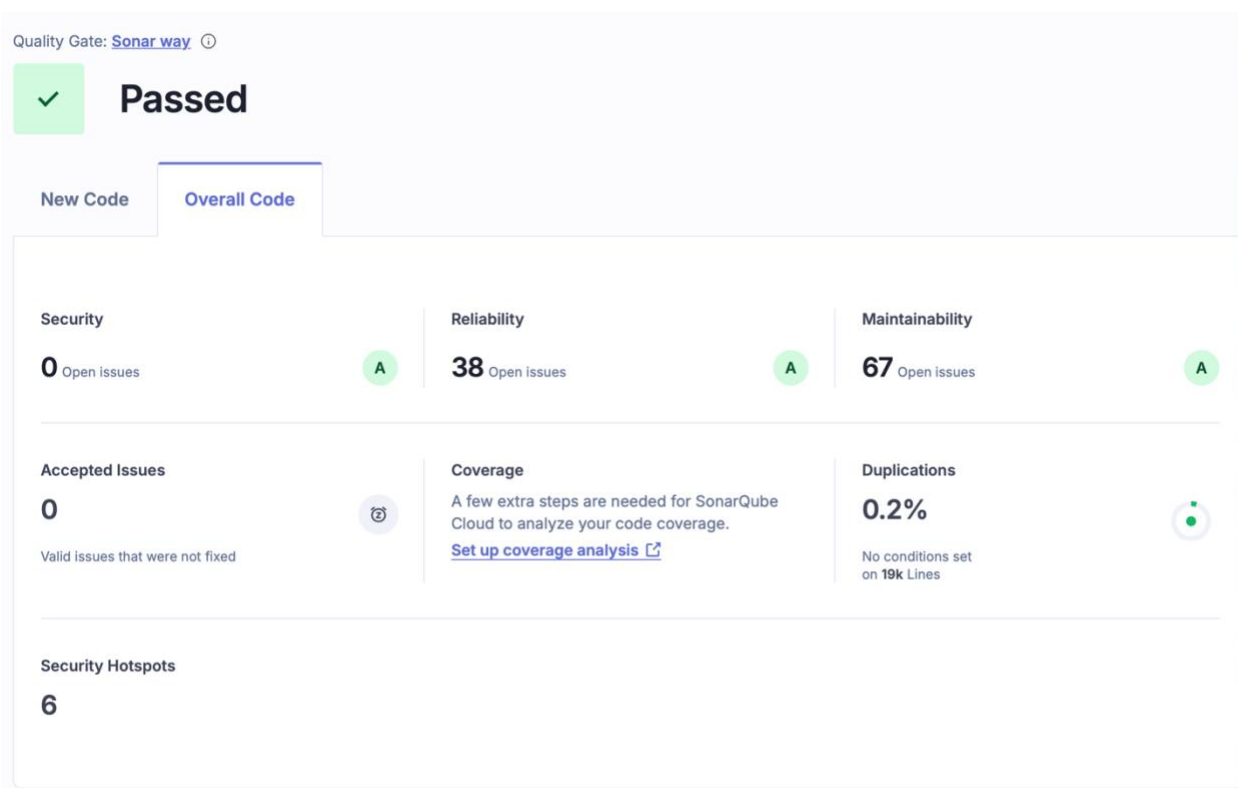


Figura 22 – SonarQube Analysis

## 5.3 Expert Review

Come richiesto dal progetto, questa sezione documenta la valutazione dell'usabilità del sistema DietiEstates25 condotta tramite Expert Review. Tale metodo non coinvolge direttamente utenti finali, ma si basa su un'ispezione sistematica dell'interfaccia da parte del team di progetto, con l'obiettivo di individuare criticità di usabilità, verificarne la coerenza e formulare proposte di miglioramento. L'approccio risulta particolarmente adatto in questa fase, poiché consente di analizzare in modo strutturato le principali funzionalità del sistema prima di una validazione più ampia con utenti potenziali. In particolare, l'analisi è stata applicata alle funzionalità di ricerca avanzata degli immobili, visualizzazione su mappa, piazzamento di un'offerta, prenotazione di una visita, cambio password, salvataggio dei preferiti e dashboard per agenti e amministratori.

L'obiettivo della review è duplice:

1. definire una checklist sistematica per la valutazione dell'usabilità;
2. applicare tale checklist per evidenziare punti di forza e criticità dell'interfaccia.

Per condurre l'ispezione è stata definita una checklist ibrida, ispirata all'impostazione del documento di riferimento, che combina tre approcci:

1. Valutazione euristica, basata principalmente sulle euristiche di Nielsen;
2. Ispezione di consistenza, per verificare uniformità terminologica, visiva e comportamentale tra le varie schermate;
3. Guidelines review, per controllare il rispetto di principi generali di chiarezza, feedback e semplicità nell'interazione.

### 5.3.1 Checklist di usabilità adottata

#### 1. Visibilità dello stato del sistema

- Il sistema fornisce un feedback chiaro dopo azioni importanti come login, cambio password, invio di un'offerta o prenotazione di una visita?
- L'utente comprende sempre in quale sezione si trova?
- Le operazioni asincrone o potenzialmente lente mostrano uno stato di caricamento o una conferma finale?

#### 2. Corrispondenza tra sistema e mondo reale

- La terminologia è coerente con il dominio immobiliare, usando espressioni come “immobile”, “annuncio”, “offerta”, “visita”, “dashboard”?
- I filtri di ricerca sono comprensibili anche per utenti non esperti?
- Le informazioni mostrate nelle card e nei dettagli immobiliari seguono un ordine logico?

#### 3. Controllo e libertà dell'utente

- L'utente può annullare o correggere facilmente un'azione?
- I form consentono modifiche prima dell'invio definitivo?
- I percorsi di navigazione evitano situazioni di blocco o disorientamento?

#### 4. Consistenza e standard

- Pulsanti, etichette e messaggi mantengono uno stile uniforme tra schermate pubbliche e schermate riservate?
- Le funzionalità disponibili cambiano in modo coerente con il ruolo dell'utente (registrato, agente, amministratore)?
- Le convenzioni di navigazione e i pattern di interazione sono ripetuti in modo prevedibile?

#### 5. Prevenzione degli errori

- I campi obbligatori sono chiaramente riconoscibili?

- Le operazioni sensibili, come il cambio password, prevedono validazioni adeguate?
- Il sistema impedisce l'invio di offerte, visite o modifiche incomplete o non valide?

## **6. Riconoscimento invece di memorizzazione**

- Le azioni principali sono immediatamente visibili?
- I filtri e le opzioni di ricerca riducono il carico mnemonico?
- Le sezioni più importanti sono riconoscibili senza che l'utente debba ricordare passaggi precedenti?

## **7. Flessibilità ed efficienza d'uso**

- La ricerca avanzata consente di arrivare rapidamente a risultati pertinenti?
- La visualizzazione su mappa supporta in modo efficace la ricerca spaziale degli immobili?
- La dashboard rende immediata l'interpretazione delle statistiche da parte di agenti e amministratori?

## **8. Design minimale ed essenziale**

- Le schermate mostrano solo le informazioni davvero utili al compito corrente?
- Le card degli immobili evitano sovraccarico informativo?
- Dashboard e form mantengono una buona leggibilità?

## **9. Gestione degli errori**

- I messaggi di errore sono chiari, localizzati e comprensibili?
- L'utente capisce come correggere il problema?
- Gli errori non interrompono inutilmente il flusso operativo?

## **10. Help e autoesplicatività**

- Il sistema è utilizzabile senza istruzioni esterne?

- Label, placeholder e messaggi guidano correttamente l'utente?
- Le operazioni più complesse rimangono comprensibili grazie alla struttura dell'interfaccia?

### 5.3.2 Esito sintetico della expert review

| ID         | Aspetto valutato                   | Esito                    | Osservazioni                                                                                                                                                                             | Miglioramento proposto                                                        |
|------------|------------------------------------|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>ER1</b> | Visibilità dello stato del sistema | Parzialmente soddisfatto | Il documento prevede notifiche di conferma ed errore per operazioni come piazzamento offerta e cambio password, ma non sempre esplicita feedback intermedi durante caricamenti o attese. | Introdurre indicatori di caricamento e messaggi di stato temporanei.          |
| <b>ER2</b> | Corrispondenza con il mondo reale  | Soddisfatto              | La terminologia adottata è coerente con il dominio immobiliare e rende chiari obiettivi e contenuti delle schermate.                                                                     | Mantenere uniformità terminologica anche nei messaggi secondari.              |
| <b>ER3</b> | Controllo e libertà dell'utente    | Parzialmente soddisfatto | I flussi principali risultano lineari, ma non sempre è evidente la presenza di azioni di annullamento o ritorno nelle operazioni più sensibili.                                          | Rafforzare i percorsi di ritorno e le azioni di correzione.                   |
| <b>ER4</b> | Consistenza e standard             | Soddisfatto              | Le funzionalità sono differenziate in base al ruolo utente e la documentazione descrive interfacce coerenti con Admin, Agente e Utente registrato.                                       | Uniformare ulteriormente posizione e denominazione dei pulsanti principali.   |
| <b>ER5</b> | Prevenzione degli errori           | Soddisfatto              | Il caso d'uso "Cambio password" include diverse validazioni: vecchia password errata, mismatch della conferma, password non conforme.                                                    | Estendere lo stesso livello di validazione ai form di prenotazione e offerta. |



|             |                                         |                          |                                                                                                                                                     |                                                                        |
|-------------|-----------------------------------------|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| <b>ER6</b>  | Riconoscimento invece di memorizzazione | Soddisfatto              | La presenza di ricerca avanzata, mappa e preferiti riduce il carico cognitivo e rende riconoscibili le azioni principali.                           | Evidenziare meglio filtri attivi e risultati selezionati.              |
| <b>ER7</b>  | Flessibilità ed efficienza              | Soddisfatto              | Ricerca avanzata e visualizzazione su mappa supportano bene utenti con obiettivi diversi, dal cercare casa al valutare opportunità immobiliari.     | Aggiungere scorciatoie o preset di filtri più frequenti.               |
| <b>ER8</b>  | Design minimale                         | Parzialmente soddisfatto | Il design è descritto come coerente e leggibile, ma dashboard e schede statistiche potrebbero diventare dense per utenti meno esperti.              | Ridurre il rumore visivo e organizzare meglio le priorità informative. |
| <b>ER9</b>  | Gestione degli errori                   | Soddisfatto              | Nei casi d'uso documentati gli errori vengono intercettati e notificati con messaggi specifici.                                                     | Rendere i messaggi ancora più orientati all'azione correttiva.         |
| <b>ER10</b> | Autoesplicatività                       | Parzialmente soddisfatto | Le funzionalità principali risultano comprensibili, ma alcune funzioni avanzate lato agente/admin possono richiedere maggiore supporto contestuale. | Inserire tooltip, testi guida brevi e placeholder più informativi.     |

### 5.3.3 Considerazioni finali della expert review

Nel complesso, la expert review evidenzia che DietiEstates25 presenta una buona base di usabilità, soprattutto in termini di coerenza col dominio applicativo, chiarezza dei casi d'uso principali e differenziazione delle funzionalità per ruolo. La parte più solida riguarda la ricerca immobiliare, il cambio password e la struttura delle funzionalità per utenti registrati, agenti e amministratori. Restano tuttavia alcuni margini di miglioramento nella visibilità dello stato del sistema, nella gestione delle operazioni asincrone e nel supporto contestuale per le funzionalità più avanzate. Tali osservazioni saranno utili come base per le successive attività di affinamento dell'interfaccia e per la valutazione con utenti potenziali.

## 5.4 Progettazione e conduzione di un esperimento con utenti reali o potenziali

A valle della expert review, è stata condotta una valutazione empirica dell'usabilità del sistema **DietiEstates25** mediante il coinvolgimento di un piccolo gruppo di utenti reali o potenziali. L'obiettivo principale dell'esperimento è stato quello di osservare il comportamento degli utenti durante l'interazione con il sistema, verificare la comprensione delle funzionalità principali e individuare eventuali criticità non emerse durante l'ispezione esperta.

L'esperimento è stato progettato in modo da coprire i principali scenari d'uso del sistema, coerentemente con le funzionalità descritte nella documentazione, tra cui: ricerca avanzata di immobili, visualizzazione su mappa, prenotazione di visite, piazzamento di offerte, gestione dei preferiti e accesso alle dashboard per utenti con privilegi differenti.

### 5.4.1 Descrizione dei soggetti coinvolti

Sono stati selezionati 5 partecipanti, scelti per rappresentare diversi profili d'uso del sistema, in modo da coprire sia utenti con bassa esperienza tecnologica sia utenti più esperti e con obiettivi specifici, come agenti immobiliari o investitori.

| Id | Età | Genere | Profilo                                                      | Esperienza con portali immobiliari | Esperienza tecnologica |
|----|-----|--------|--------------------------------------------------------------|------------------------------------|------------------------|
| U1 | 24  | M      | Giovane lavoratore in cerca della prima casa                 | Media                              | Alta                   |
| U2 | 34  | F      | Madre lavoratrice interessata a servizi e vicinanza a scuole | Bassa                              | Media                  |
| U3 | 46  | M      | Agente immobiliare                                           | Alta                               | Alta                   |
| U4 | 58  | F      | Utente occasionale con poca familiarità digitale             | Bassa                              | Bassa                  |
| U5 | 39  | M      | Investitore interessato a offerte e statistiche              | Alta                               | Alta                   |

### 5.4.2 Definizione dei task

A ciascun partecipante sono stati assegnati uno o più task rappresentativi delle funzionalità principali del sistema.

| Utente | Task                                                                                                                   |
|--------|------------------------------------------------------------------------------------------------------------------------|
| U1     | Effettuare una ricerca avanzata filtrando per prezzo, località e numero di stanze; salvare un immobile tra i preferiti |
| U2     | Cercare un immobile utilizzando la mappa; prenotare una visita                                                         |
| U3     | Accedere alla dashboard agente e interpretare i dati sugli immobili                                                    |
| U4     | Visualizzare un immobile e modificare la password                                                                      |
| U5     | Visualizzare un immobile, piazzare un'offerta e analizzare i dati nella dashboard                                      |

I task sono stati progettati per coprire:

- operazioni di consultazione (ricerca, visualizzazione),
- operazioni interattive (prenotazione, offerta),
- operazioni gestionali (dashboard, gestione account).

### 5.4.3 Procedura sperimentale

L'esperimento è stato condotto in modalità controllata, con ciascun partecipante coinvolto singolarmente. Prima dell'inizio, agli utenti è stata fornita una breve introduzione sul contesto applicativo, senza però spiegare nel dettaglio le funzionalità del sistema, al fine di osservare un'interazione il più possibile spontanea.

Durante l'esecuzione dei task:

- agli utenti è stato chiesto di pensare ad alta voce, descrivendo ciò che stavano facendo e le eventuali difficoltà incontrate;
- è stato osservato il comportamento dell'utente senza intervenire direttamente, salvo nei casi di blocco totale;
- sono stati annotati errori, esitazioni, tempi di esecuzione e commenti spontanei.

Al termine dei task, è stato chiesto a ciascun partecipante di esprimere un giudizio complessivo sull'esperienza d'uso.

### 5.4.4 Metriche raccolte

Per ogni task sono stati considerati i seguenti indicatori:

- **Successo del task** (completato / parzialmente completato / non completato)
- **Tempo di esecuzione**
- **Numero di errori**
- **Livello di soddisfazione percepita**
- **Osservazioni qualitative**

### 5.4.5 Risultati dell'esperimento

| Utente | Task completato | Tempo | Errori | Osservazioni                                                                                  |
|--------|-----------------|-------|--------|-----------------------------------------------------------------------------------------------|
| U1     | Sì              | Medio | Bassi  | Ha completato facilmente la ricerca avanzata e l'aggiunta ai preferiti; interfaccia intuitiva |
| U2     | Parzialmente    | Alto  | Medi   | Difficoltà iniziale nell'uso della mappa e nella prenotazione                                 |
| U3     | Sì              | Basso | Bassi  | Navigazione fluida nella dashboard; esperienza positiva                                       |
| U4     | Parzialmente    | Alto  | Alti   | Difficoltà nella comprensione delle azioni e dei form                                         |
| U5     | Sì              | Medio | Bassi  | Offerta e dashboard comprensibili, ma alcune informazioni poco evidenti                       |

### 5.4.6 Analisi dei risultati

Dall'analisi dei dati raccolti emergono alcuni aspetti rilevanti:

- La ricerca avanzata risulta efficace e intuitiva per utenti con esperienza media o alta;
- Le funzionalità principali (visualizzazione immobili, preferiti, offerte) sono generalmente comprensibili;

- La dashboard è apprezzata dagli utenti più esperti, che riescono a interpretare facilmente i dati.
  - Gli utenti con bassa esperienza tecnologica incontrano difficoltà nell'utilizzo della mappa e dei form;
  - Alcune operazioni, come la prenotazione o il cambio password, non forniscono sempre feedback immediato;
  - La quantità di informazioni nelle dashboard può risultare elevata per utenti non esperti.
- 

### 5.4.7 Considerazioni finali

L'esperimento con utenti reali o potenziali ha confermato in larga parte i risultati emersi durante la expert review. Il sistema risulta complessivamente usabile e coerente, soprattutto per utenti con una minima familiarità con strumenti digitali e portali immobiliari.

Tuttavia, sono emerse alcune criticità legate alla semplicità d'uso per utenti meno esperti e alla necessità di migliorare la visibilità dello stato del sistema durante alcune operazioni. Tali aspetti rappresentano possibili direzioni di miglioramento per evoluzioni future del sistema.